UNIVERSITE IBN ZOHR
FACULTE DES SCIENCES
AGADIR

جامعة ابن زهر
كليـــة العلـــوم
أكاديـر

Algorithmique et Programmation (SMP3)
TD1 : Instructions élémentaires
2024-2025

Exercice 5
Donner le contenu de la variable X après la série des actions suivantes : Variable
X : Entier
Début
X←5
X + 1 ← X mod(3)
Fin
Solution
1. Initialisation de la variable X
Début : On assigne à X la valeur 5.
X ← 5
2. Instruction suivante : X + 1 ← X mod(3)
Cette instruction est inhabituelle car elle semble incorrecte sur le plan syntaxique. Normalement, en algorithme, une telle expression n'est pas courante, car X + 1 ne peut pas être affecté à une valeur.
Si l'objectif était de calculer le modulo et d'affecter le résultat à X, cela devrait probablement être :
X ← X mod(3)
Appliquons cela :
X mod(3) signifie le reste de la division de X par 3.
5 mod(3) = 2 (car 5 divisé par 3 donne un quotient de 1 et un reste de 2)
Donc, à la fin de cette étape :
X ← 2

Exercice 6
Écrire l'algorithme qui lit un entier n, puis calcule et affiche son double et son carré. Ensuite, rédige le code en Python correspondant.
Solution
```
# Lecture de l'entier n
n = int(input("Entrez un entier : "))

# Calcul du double et du carré
double_n = 2 * n
carre_n = n * n

# Affichage des résultats
print(f"Le double de {n} est : {double_n}")
print(f"Le carré de {n} est : {carre_n}")
```
Explication :
Lecture de l'entier : On utilise la fonction input() pour demander à l'utilisateur d'entrer un entier, puis on le convertit en un entier avec int().
Calculs :
Le double de n
n est calculé en multipliant n n par 2.
Le carré de n
n est calculé en multipliant n n par lui-même.
Affichage : Les résultats du double et du carré sont affichés à l'aide de la fonction print().

Exercice 7
Écrire un algorithme qui lit le prix hors taxe d'un produit, le nombre de produits achetés, le taux de TVA et calcule le prix total TTC. Ensuite, rédige le code en Python correspondant.
Solution
```
# Lecture des données
prix_ht = float(input("Entrez le prix hors taxe (HT) du produit : "))
nombre_produits = int(input("Entrez le nombre de produits achetés : "))
taux_tva = float(input("Entrez le taux de TVA (en pourcentage) : "))

# Calcul de la TVA pour un produit
tva = prix_ht * (taux_tva / 100)

# Calcul du prix total TTC
prix_ttc_par_produit = prix_ht + tva
prix_total_ttc = prix_ttc_par_produit * nombre_produits

# Affichage du résultat
print(f"Le prix total TTC est : {prix_total_ttc:.2f} euros")
```
Explication :
Lecture des données :
Le prix HT (hors taxe) du produit est lu en tant que nombre à virgule flottante (float).
Le nombre de produits achetés est un entier.
Le taux de TVA est lu en pourcentage et est également converti en nombre flottant.
Calculs :
On calcule la TVA pour un produit en multipliant le prix hors taxe par le taux de TVA divisé par 100.
Le prix TTC pour un produit est la somme du prix hors taxe et de la TVA.
Le prix total TTC est ensuite obtenu en multipliant le prix TTC par le nombre de produits achetés.
Affichage : Le prix total TTC est affiché avec deux décimales pour représenter le montant en euros de manière claire et précise.

Exercice 8
Écrire un algorithme qui permet de calculer le nombre de radiateurs dont on a besoin pour chauffer une pièce. Ensuite, rédige le code en Python correspondant.
Notons qu'un radiateur est capable de chauffer 8m^3.
L'utilisateur donnera la longueur, la largeur et la hauteur de la pièce en mètres.
Solution
```
import math

# Lecture des dimensions de la pièce
longueur = float(input("Entrez la longueur de la pièce en mètres : "))
largeur = float(input("Entrez la largeur de la pièce en mètres : "))
hauteur = float(input("Entrez la hauteur de la pièce en mètres : "))

# Calcul du volume de la pièce
```

```
volume_piece = longueur * largeur * hauteur

# Un radiateur peut chauffer 8 m³
volume_par_radiateur = 8

# Calcul du nombre de radiateurs nécessaires (arrondi à l'entier supérieur)
nombre_radiateurs = math.ceil(volume_piece / volume_par_radiateur)

# Affichage du résultat
print(f"Le nombre de radiateurs nécessaires pour chauffer la pièce est : {nombre_radiateurs}")
```

Explication :
Lecture des dimensions : Nous demandons à l'utilisateur d'entrer les dimensions de la pièce (longueur, largeur, hauteur).
Calcul du volume : On calcule le volume en multipliant la longueur, la largeur et la hauteur.
Calcul du nombre de radiateurs : On divise le volume de la pièce par 8 (car un radiateur peut chauffer 8 m³). Ensuite, on utilise math.ceil() pour arrondir à l'entier supérieur, car même si une petite partie de la pièce reste à chauffer, un radiateur supplémentaire sera nécessaire.
Affichage : On affiche le nombre total de radiateurs requis.

Exercice 9

Écrire un programme utilisant des variables de type chaine de caractères et affichant, en utilisant l'opération de concaténation, quatre variantes possibles des chaînes suivantes « rose », « nulle », « épine », « sans ». Ensuite, rédige le code en Python correspondant.

Solution

```
# Déclaration des chaînes
chaine1 = "rose"
chaine2 = "nulle"
chaine3 = "épine"
chaine4 = "sans"

# Variantes possibles en utilisant la concaténation correcte
variante1 = chaine1 + " " + chaine2 + " " + chaine3 + " " + chaine4
variante2 = chaine4 + " " + chaine1 + " " + chaine2 + " " + chaine3
 variante3 = chaine4 + " " + chaine3 + " " + chaine1 + " " + chaine2
 variante4 = chaine2 + " " + chaine3 + " " + chaine4 + " " + chaine1
# Affichage des résultats
print("Les variantes possibles sont :")
print(variante1)
print(variante2)
print(variante3)
print(variante4)
```

Explication :
Déclaration des chaînes : On crée quatre variables, chacune contenant une des chaînes fournies.
Concaténation :
On utilise l'opérateur + pour concaténer les chaînes, et on ajoute des espaces entre les mots pour assurer une présentation correcte.
Affichage : On affiche les quatre variantes possibles obtenues par concaténation.

Exercice 10

Écrire un algorithme pour convertir un nombre de secondes en un nombre d'heures, de minutes et de secondes. On utilisera les opérateurs modulo et division entière. Ensuite, rédige le code en Python correspondant.

Solution

```
# Lecture du nombre total de secondes
total_secondes = int(input("Entrez le nombre total de secondes : "))

# Calcul du nombre d'heures
heures = total_secondes // 3600

# Calcul du nombre de minutes restantes
minutes = (total_secondes % 3600) // 60

# Calcul des secondes restantes
secondes = total_secondes % 60

# Affichage du résultat
print(f"{total_secondes} secondes correspondent à :")
print(f"{heures} heures, {minutes} minutes et {secondes} secondes")
```

Explication :
Lecture des secondes : On demande à l'utilisateur de saisir un nombre entier représentant le total de secondes.
Calcul des heures : On divise le nombre total de secondes par 3600 pour obtenir le nombre d'heures (en utilisant la division entière //).
Calcul des minutes : On utilise l'opérateur modulo (%) pour obtenir le reste des secondes après avoir retiré les heures, puis on divise ce reste par 60 pour obtenir les minutes.
Calcul des secondes restantes : Le reste des secondes après le calcul des heures et des minutes donne les secondes restantes.
Affichage : Le résultat est affiché sous la forme "heures, minutes, secondes".

Exemple

3671 secondes correspondent à : 1 heure, 1 minute et 11 secondes

Exercice 11

Écrire un algorithme qui calcule :
- le périmètre et la surface d'un rectangle.
- le périmètre et la surface d'un cercle.
- la surface latérale et le volume d'un cylindre. - la surface s et le volume v d'une sphère.
Ensuite, rédige le code en Python correspondant.

- **Solution**

```
import math

# 1. Calcul du périmètre et de la surface d'un rectangle
longueur = float(input("Entrez la longueur du rectangle : "))
largeur = float(input("Entrez la largeur du rectangle : "))
perimetre_rectangle = 2 * (longueur + largeur)
surface_rectangle = longueur * largeur
print(f"Périmètre du rectangle : {perimetre_rectangle}")
print(f"Surface du rectangle : {surface_rectangle}")

# 2. Calcul du périmètre et de la surface d'un cercle
rayon_cercle = float(input("Entrez le rayon du cercle : "))
perimetre_cercle = 2 * math.pi * rayon_cercle
surface_cercle = math.pi * rayon_cercle**2
print(f"Périmètre du cercle : {perimetre_cercle}")
print(f"Surface du cercle : {surface_cercle}")

# 3. Calcul de la surface latérale et du volume d'un cylindre
rayon_cylindre = float(input("Entrez le rayon de la base du cylindre : "))
hauteur_cylindre = float(input("Entrez la hauteur du cylindre : "))
surface_laterale_cylindre = 2 * math.pi * rayon_cylindre * hauteur_cylindre
volume_cylindre = math.pi * rayon_cylindre**2 * hauteur_cylindre
print(f"Surface latérale du cylindre : {surface_laterale_cylindre}")
print(f"Volume du cylindre : {volume_cylindre}")
```

```
# 4. Calcul de la surface et du volume d'une sphère
rayon_sphere = float(input("Entrez le rayon de la sphère : "))
surface_sphere = 4 * math.pi * rayon_sphere**2
volume_sphere = (4/3) * math.pi * rayon_sphere**3
print(f"Surface de la sphère : {surface_sphere}")
print(f"Volume de la sphère : {volume_sphere}")
```

Explication :

1. **Rectangle :**
   - Le périmètre d'un rectangle est calculé par la formule P=2×(longueur+largeur)P = 2 \times (longueur + largeur)P=2×(longueur+largeur).
   - La surface est calculée par la formule S=longueur×largeurS = longueur \times largeurS=longueur×largeur.
2. **Cercle :**
   - Le périmètre d'un cercle (circonférence) est donné par P=2×π×rayonP = 2 \times \pi \times rayonP=2×π×rayon.
   - La surface d'un cercle est calculée par S=π×rayon2S = \pi \times rayon^2S=π×rayon2.
3. **Cylindre :**
   - La surface latérale du cylindre est calculée par Slate´rale=2×π×rayon×hauteurS_{latérale} = 2 \times \pi \times rayon \times hauteurSlate´rale=2×π×rayon×hauteur.
   - Le volume est calculé par V=π×rayon2×hauteurV = \pi \times rayon^2 \times hauteurV=π×rayon2×hauteur.
4. **Sphère :**
   - La surface d'une sphère est donnée par S=4×π×rayon2S = 4 \times \pi \times rayon^2S=4×π×rayon2.
   - Le volume d'une sphère est calculé par V=43×π×rayon3V = \frac{4}{3} \times \pi \times rayon^3V=34×π×rayon3.

Exemple d'exécution :

Si l'utilisateur entre des valeurs pour un rectangle, cercle, cylindre, et sphère, le programme affiche les résultats correspondants pour le périmètre, surface et volume des objets géométriques.

UNIVERSITE IBN ZOHR
FACULTE DES SCIENCES
AGADIR

جامعة ابن زهر
كليـــة العلـــوم
أكاديـر

Algorithmique et Programmation (SMP3)
TD1 : Instructions élémentaires
2024-2025

Exercice 9
Écrire un programme utilisant des variables de type chaine de caractères et affichant, en utilisant l'opération de concaténation, quatre variantes possibles des chaînes suivantes « rose », « nulle », « épine », « sans ». Ensuite, rédige le code en Python correspondant.
Solution

```
# Déclaration des chaînes
chaine1 = "rose"
chaine2 = "nulle"
chaine3 = "épine"
chaine4 = "sans"

# Variantes possibles en utilisant la concaténation correcte
variante1 = chaine1 + " " + chaine2 + " " + chaine3 + " " + chaine4
variante2 = chaine4 + " " + chaine1 + " " + chaine2 + " " + chaine3
 variante3 = chaine4 + " " + chaine3 + " " + chaine1 + " " + chaine2
 variante4 = chaine2 + " " + chaine3 + " " + chaine4 + " " + chaine1
# Affichage des résultats
print("Les variantes possibles sont :")
print(variante1)
print(variante2)
print(variante3)
print(variante4)
```

Explication :
Déclaration des chaînes : On crée quatre variables, chacune contenant une des chaînes fournies.
Concaténation :
On utilise l'opérateur + pour concaténer les chaînes, et on ajoute des espaces entre les mots pour assurer une présentation correcte.
Affichage : On affiche les quatre variantes possibles obtenues par concaténation.
Exercice 10
Écrire un algorithme pour convertir un nombre de secondes en un nombre d'heures, de minutes et de secondes. On utilisera les opérateurs modulo et division entière. Ensuite, rédige le code en Python correspondant.
Solution

```
# Lecture du nombre total de secondes
total_secondes = int(input("Entrez le nombre total de secondes : "))

# Calcul du nombre d'heures
heures = total_secondes // 3600

# Calcul du nombre de minutes restantes
minutes = (total_secondes % 3600) // 60

# Calcul des secondes restantes
secondes = total_secondes % 60

# Affichage du résultat
print(f"{total_secondes} secondes correspondent à :")
print(f"{heures} heures, {minutes} minutes et {secondes} secondes")
```

Explication :
Lecture des secondes : On demande à l'utilisateur de saisir un nombre entier représentant le total de secondes.
Calcul des heures : On divise le nombre total de secondes par 3600 pour obtenir le nombre d'heures (en utilisant la division entière //).
Calcul des minutes : On utilise l'opérateur modulo (%) pour obtenir le reste des secondes après avoir retiré les heures, puis on divise ce reste par 60 pour obtenir les minutes.
Calcul des secondes restantes : Le reste des secondes après le calcul des heures et des minutes donne les secondes restantes.
Affichage : Le résultat est affiché sous la forme "heures, minutes, secondes".
Exemple
3671 secondes correspondent à : 1 heure, 1 minute et 11 secondes
Exercice 11
Écrire un algorithme qui calcule :
- le périmètre et la surface d'un rectangle.
- le périmètre et la surface d'un cercle.
- la surface latérale et le volume d'un cylindre. - la surface s et le volume v d'une sphère.
Ensuite, rédige le code en Python correspondant.
- **Solution**

```
import math

# 1. Calcul du périmètre et de la surface d'un rectangle
longueur = float(input("Entrez la longueur du rectangle : "))
largeur = float(input("Entrez la largeur du rectangle : "))
perimetre_rectangle = 2 * (longueur + largeur)
surface_rectangle = longueur * largeur
print(f"Périmètre du rectangle : {perimetre_rectangle}")
print(f"Surface du rectangle : {surface_rectangle}")

# 2. Calcul du périmètre et de la surface d'un cercle
rayon_cercle = float(input("Entrez le rayon du cercle : "))
perimetre_cercle = 2 * math.pi * rayon_cercle
surface_cercle = math.pi * rayon_cercle**2
print(f"Périmètre du cercle : {perimetre_cercle}")
print(f"Surface du cercle : {surface_cercle}")

# 3. Calcul de la surface latérale et du volume d'un cylindre
rayon_cylindre = float(input("Entrez le rayon de la base du cylindre : "))
```

```
hauteur_cylindre = float(input("Entrez la hauteur du cylindre : "))
surface_laterale_cylindre = 2 * math.pi * rayon_cylindre * hauteur_cylindre
volume_cylindre = math.pi * rayon_cylindre**2 * hauteur_cylindre
print(f"Surface latérale du cylindre : {surface_laterale_cylindre}")
print(f"Volume du cylindre : {volume_cylindre}")

# 4. Calcul de la surface et du volume d'une sphère
rayon_sphere = float(input("Entrez le rayon de la sphère : "))
surface_sphere = 4 * math.pi * rayon_sphere**2
volume_sphere = (4/3) * math.pi * rayon_sphere**3
print(f"Surface de la sphère : {surface_sphere}")
print(f"Volume de la sphère : {volume_sphere}")
```

Explication :

1. **Rectangle :**
   - Le périmètre d'un rectangle est calculé par la formule P=2×(longueur+largeur)P = 2 \times (longueur + largeur)P=2×(longueur+largeur).
   - La surface est calculée par la formule S=longueur×largeurS = longueur \times largeurS=longueur×largeur.
2. **Cercle :**
   - Le périmètre d'un cercle (circonférence) est donné par P=2×π×rayonP = 2 \times \pi \times rayonP=2×π×rayon.
   - La surface d'un cercle est calculée par S=π×rayon2S = \pi \times rayon^2S=π×rayon2.
3. **Cylindre :**
   - La surface latérale du cylindre est calculée par Slate´rale=2×π×rayon×hauteurS_{latérale} = 2 \times \pi \times rayon \times hauteurSlate´rale=2×π×rayon×hauteur.
   - Le volume est calculé par V=π×rayon2×hauteurV = \pi \times rayon^2 \times hauteurV=π×rayon2×hauteur.
4. **Sphère :**
   - La surface d'une sphère est donnée par S=4×π×rayon2S = 4 \times \pi \times rayon^2S=4×π×rayon2.
   - Le volume d'une sphère est calculé par V=43×π×rayon3V = \frac{4}{3} \times \pi \times rayon^3V=34×π×rayon3.

Exemple d'exécution :

Si l'utilisateur entre des valeurs pour un rectangle, cercle, cylindre, et sphère, le programme affiche les résultats correspondants pour le périmètre, surface et volume des objets géométriques.

Exercice 1

Écrire un algorithme qui demande deux nombres à l'utilisateur et l'informe ensuite si leur produit est négatif, positif ou nul. Ensuite, rédige le code en Python correspondant.

Algorithme SigneProduit Variables a, b : Reel Début
Ecrire ("Entrez deux nombre : ") Lire (a, b)
Si ((a > 0 ET b > 0) OU (a < 0 ET b < 0)) Alors
Ecrire ("Le produit est positif")
SinonSi (a = 0 OU b = 0) Alors Ecrire ("Le produit est nul")
Fin
Sinon FinSi

Ecrire ("Le produit est négatif")

```
# Demander les deux nombres à l'utilisateur
nombre1 = float(input("Entrez le premier nombre : "))
nombre2 = float(input("Entrez le deuxième nombre : "))

# Calculer le produit
produit = nombre1 * nombre2

# Vérifier la nature du produit
if produit > 0:
    print("Le produit est positif.")
elif produit < 0:
    print("Le produit est négatif.")
else:
    print("Le produit est nul.")
```

Exercice 2

Ecrire un algorithme qui demande trois noms à l'utilisateur et l'informe ensuite s'ils sont rangés ou non dans l'ordre alphabétique. Ensuite, rédige le code en Python correspondant.

Algorithme OrdreNoms Variables a, b, c : Chaine Début
Ecrire ("Entrez successivement trois noms : ") Lire (a, b, c)
Si (a <= b ET b <= c) Alors
Ecrire ("Ces noms sont classés alphabétiquement")
Fin
Sinon FinSi

Ecrire ("Ces noms ne sont pas classés")

```
# Demander les trois noms à l'utilisateur
nom1 = input("Entrez le premier nom : ")
nom2 = input("Entrez le deuxième nom : ")
nom3 = input("Entrez le troisième nom : ")

# Vérifier si les noms sont dans l'ordre alphabétique
if nom1 <= nom2 <= nom3:
    print("Les noms sont dans l'ordre alphabétique.")
else:
    print("Les noms ne sont pas dans l'ordre alphabétique.")
```

Exercice 3

Donner le résultat de l'algorithme suivant en saisissant successivement au clavier les valeurs 20, 150, 50, 51 et 400.

Algorithme résultat_si Variable valeur : entier Début
Lire (valeur)
Si (valeur > 100) alors
Ecrire "A"
Si (valeur > 200) alors
Sinon
Sinon FinSi
Ecrire "B" Ecrire "C"
Si (valeur > 50) alors
Ecrire "D"
Fin
FinSi
Sinon FinSi
Ecrire "E"
Données Sorties de l'algorithme (impression) 20 : E
150 : A C
50 : E

51 : D
400 : A B

Exercice 4

Ecrire un algorithme qui demande l'âge d'un enfant à l'utilisateur. Ensuite, il l'informe de sa catégorie:
"Poussin" de 6 à 7 ans
"Pupille" de 8 à 9 ans
"Minime" de 10 à 11 ans "Cadet" après 12 ans

```
Algorithme CategorieAge Variable age :Entier Début
Ecrire ("Entrez l'âge de l'enfant : ") Lire (age)
Si (age >= 12) Alors
Ecrire ("Catégorie Cadet") SinonSi (age >= 10) Alors
Ecrire ("Catégorie Minime") SinonSi (age >= 8) Alors
Ecrire ("Catégorie Pupille") SinonSi (age >= 6) Alors
Ecrire ("Catégorie Poussin")
Finsi
Fin
# Demander l'âge de l'enfant
age = int(input("Entrez l'âge de l'enfant : "))

# Vérifier la catégorie en fonction de l'âge
if age >= 12:
    print("Catégorie Cadet")
elif age >= 10:
    print("Catégorie Minime")
elif age >= 8:
    print("Catégorie Pupille")
elif age >= 6:
    print("Catégorie Poussin")
else:
    print("Aucune catégorie définie pour cet âge.")
```

Exercice5

Ecrire un algorithme qui lit trois valeurs entières (A, B et C) et qui permet de les trier par échanges successifs. Enfin, il les affiche dans l'ordre.

```
Algorithme TriSuccessif
Variables A, B, C, Z : Entier

Debut
 Ecrire (" entrer Les valeurs de A, B et C") Lire (A, B, C)
Si (A > B) ALORS
Z ← A
A ← B
B ← Z
 FinSi
Si (B > C) ALORS
Z ← B
B ← C
C ← Z
Si (A > B) ALORS
Z ← A
A ← B
B ← Z
FinSi
FinSi
Ecrire ("Les valeurs A , B et C sont (dans l'ordre):", A , " ", B, " ", C)
Fin
# Demander les trois valeurs à l'utilisateur
A = int(input("Entrez la valeur de A : "))
B = int(input("Entrez la valeur de B : "))
C = int(input("Entrez la valeur de C : "))

# Premier échange entre A et B si nécessaire
if A > B:
    A, B = B, A

# Deuxième échange entre B et C si nécessaire
if B > C:
    B, C = C, B

    # Vérifier de nouveau A et B après l'échange
    if A > B:
        A, B = B, A

# Afficher les valeurs triées
print("Les valeurs A, B et C dans l'ordre croissant sont :", A, B, C)
```

Algorithmique et Programmation (SMP3)
TD2 : Instructions élémentaires
2024-2025

.

Exercice 1

Écrire un algorithme qui demande deux nombres à l'utilisateur et l'informe ensuite si leur produit est négatif, positif ou nul. Ensuite, rédige le code en Python correspondant.

Algorithme SigneProduit Variables a, b : Reel Début
Ecrire ("Entrez deux nombre : ") Lire (a, b)
Si ((a > 0 ET b > 0) OU (a < 0 ET b < 0)) Alors
Ecrire ("Le produit est positif")
SinonSi (a = 0 OU b = 0) Alors Ecrire ("Le produit est nul")
Fin
Sinon FinSi

Ecrire ("Le produit est négatif")

python

```
# Demander les deux nombres à l'utilisateur
nombre1 = float(input("Entrez le premier nombre : "))
nombre2 = float(input("Entrez le deuxième nombre : "))

# Calculer le produit
produit = nombre1 * nombre2

# Vérifier la nature du produit
if produit > 0:
    print("Le produit est positif.")
elif produit < 0:
    print("Le produit est négatif.")
else:
    print("Le produit est nul.")
```

Exercice 2

Ecrire un algorithme qui demande trois noms à l'utilisateur et l'informe ensuite s'ils sont rangés ou non dans l'ordre alphabétique. Ensuite, rédige le code en Python correspondant.

Algorithme OrdreNoms Variables a, b, c : Chaine Début
Ecrire ("Entrez successivement trois noms : ") Lire (a, b, c)
Si (a <= b ET b <= c) Alors
Ecrire ("Ces noms sont classés alphabétiquement")
Fin
Sinon FinSi

Python

```
Ecrire ("Ces noms ne sont pas classés")
# Demander les trois noms à l'utilisateur
nom1 = input("Entrez le premier nom : ")
nom2 = input("Entrez le deuxième nom : ")
nom3 = input("Entrez le troisième nom : ")

# Vérifier si les noms sont dans l'ordre alphabétique
if nom1 <= nom2 <= nom3:
    print("Les noms sont dans l'ordre alphabétique.")
else:
    print("Les noms ne sont pas dans l'ordre alphabétique.")
```

Exercice 3

Donner le résultat de l'algorithme suivant en saisissant successivement au clavier les valeurs 20, 150, 50, 51 et 400.

Algorithme résultat_si Variable valeur : entier Début
Lire (valeur)
Si (valeur > 100) alors
Ecrire "A"
Si (valeur > 200) alors
Sinon
Sinon FinSi
Ecrire "B" Ecrire "C"
Si (valeur > 50) alors
Ecrire "D"
Fin
FinSi
Sinon FinSi
Ecrire "E"
Données Sorties de l'algorithme (impression) 20 : E
150 : A C
50 : E
51 : D
400 : A B

Exercice 4

Ecrire un algorithme qui demande l'âge d'un enfant à l'utilisateur. Ensuite, il l'informe de sa catégorie:
"Poussin" de 6 à 7 ans
"Pupille" de 8 à 9 ans
"Minime" de 10 à 11 ans "Cadet" après 12 ans
Algorithme CategorieAge Variable age :Entier Début
Ecrire ("Entrez l'âge de l'enfant : ") Lire (age)
Si (age >= 12) Alors
Ecrire ("Catégorie Cadet") SinonSi (age >= 10) Alors
Ecrire ("Catégorie Minime") SinonSi (age >= 8) Alors
Ecrire ("Catégorie Pupille") SinonSi (age >= 6) Alors
Ecrire ("Catégorie Poussin")
Finsi
Fin

Python

```
# Demander l'âge de l'enfant
age = int(input("Entrez l'âge de l'enfant : "))

# Vérifier la catégorie en fonction de l'âge
if age >= 12:
    print("Catégorie Cadet")
elif age >= 10:
    print("Catégorie Minime")
elif age >= 8:
    print("Catégorie Pupille")
elif age >= 6:
    print("Catégorie Poussin")
else:
    print("Aucune catégorie définie pour cet âge.")
```

Exercice5

Ecrire un algorithme qui lit trois valeurs entières (A, B et C) et qui permet de les trier par échanges successifs. Enfin, il les affiche dans l'ordre.

```
Algorithme TriSuccessif
Variables A, B, C, Z : Entier

Debut
 Ecrire (" entrer Les valeurs de A, B et C") Lire (A, B, C)
Si (A > B) ALORS
Z ← A
A ← B
B ← Z
 FinSi
Si (B > C) ALORS
Z ← B
B ← C
C ← Z
Si (A > B) ALORS
Z ← A
A ← B
B ← Z
FinSi
FinSi
Ecrire ("Les valeurs A , B et C sont (dans l'ordre):", A , " ", B, " ", C)
Fin
```

Python

```
# Demander les trois valeurs à l'utilisateur
A = int(input("Entrez la valeur de A : "))
B = int(input("Entrez la valeur de B : "))
C = int(input("Entrez la valeur de C : "))

# Premier échange entre A et B si nécessaire
if A > B:
    A, B = B, A

# Deuxième échange entre B et C si nécessaire
if B > C:
    B, C = C, B

# Vérifier de nouveau A et B après l'échange
if A > B:
    A, B = B, A

# Afficher les valeurs triées
print("Les valeurs A, B et C dans l'ordre croissant sont :", A, B, C)
```

Exercice 6

Ecrire un algorithme permettant de résoudre une équation du second degré.

Ecrire un algorithme permettant de résoudre une équation du second degré.

```
Algorithme SecondDegre
Variables a, b, c, delta : Reel
Debut
        Ecrire (" saisissez les valeurs a, b et c de l'équation ax²+bx+c=0 : ")
        Lire (a, b, c)
        Si (a=0) Alors
                Ecrire (" équation du premier degré ")
                Si (b <> 0) ALORS
                        Ecrire ("La solution est ", -c/b)
                SinonSi (c = 0) Alors
                        Ecrire (" Tous les nombres vérifient l'équation ")
                Sinon  Ecrire (" Pas de solution")
                FinSi
        Sinon
                delta ← b*b-4*a*c
                Si (delta > 0) Alors
                        Ecrire ("La première solution est :  ", (-b-sqrt(delta ))/(2*a) )
                        Ecrire ("La deuxième solution est : ", (-b+sqrt(delta ))/(2*a) )
                SinonSi (delta =0) Alors
                        Ecrire ("La solution est : ", -b/(2a))
                Sinon Ecrire ("pas de solutions réelles !!")
                FinSi
        FinSi
Fin
```

Python

```
import math

# Demander les coefficients a, b et c
a = float(input("Saisissez la valeur de a : "))
b = float(input("Saisissez la valeur de b : "))
c = float(input("Saisissez la valeur de c : "))

# Cas d'une équation du premier degré si a = 0
if a == 0:
    print("Équation du premier degré.")
    if b != 0:
        print("La solution est :", -c / b)
    elif c == 0:
        print("Tous les nombres vérifient l'équation.")
    else:
        print("Pas de solution.")
else:
```

```
# Calcul du discriminant (delta)
delta = b ** 2 - 4 * a * c

# Cas où delta est positif : deux solutions réelles distinctes
if delta > 0:
    x1 = (-b - math.sqrt(delta)) / (2 * a)
    x2 = (-b + math.sqrt(delta)) / (2 * a)
    print("La première solution est :", x1)
    print("La deuxième solution est :", x2)

# Cas où delta est nul : une solution réelle unique
elif delta == 0:
    x = -b / (2 * a)
    print("La solution est :", x)

# Cas où delta est négatif : pas de solutions réelles
else:
    print("Pas de solutions réelles !!")
```

Explication des différentes parties :

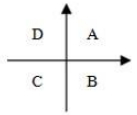
1. **Lecture des coefficients `a`, `b`, et `c`** : On demande à l'utilisateur de saisir les valeurs de ces coefficients.
2. **Vérification si `a = 0`** : Si `a` est nul, l'équation devient une équation du premier degré (`bx + c = 0`). On résout cette équation avec les sous-cas :
   - Si `b ≠ 0`, il existe une solution : x=−cbx = -\frac{c}{b}x=−bc.
   - Si `b = 0` et `c = 0`, tous les nombres sont solutions (équation identique).
   - Si `b = 0` et `c ≠ 0`, il n'y a pas de solution.
3. **Calcul du discriminant `delta` pour une équation du second degré** :
   - Si `delta > 0`, il y a deux solutions réelles distinctes.
   - Si `delta == 0`, il y a une solution réelle unique.
   - Si `delta < 0`, il n'y a pas de solutions réelles.

Exemples d'utilisation :

- Pour l'équation x2−3x+2=0x^2 - 3x + 2 = 0x2−3x+2=0, entrez `a = 1`, `b = -3`, `c = 2`. Les solutions sont 1 et 2.
- Pour une équation du premier degré 0×x2+2x−4=00 \times x^2 + 2x - 4 = 00×x2+2x−4=0, entrez `a = 0`, `b = 2`, `c = -4`. La solution est 2.

Exercice 7

Écrire un algorithme qui, étant donné les coordonnées x et y d'un point, détermine dans quelle partie (A, B, C ou D) du plan se trouve le point (cf. figure1).




```
Algorithme PointPlan
Variables x, y : Reel
Debut
        Ecrire ("Saisir les coordonnées du point")
        Lire (x, y)
        Si (x > 0) Alors
                Si(y > 0) Alors Ecrire ("A")
                SinonSi (y < 0) Alors Ecrire ("B")
                Sinon Ecrire ("AB")
                FinSi
        SinonSi (x < 0) Alors
                Si(y > 0) Alors Ecrire ("D")
                SinonSi (y < 0) Alors Ecrire ("C")
                Sinon Ecrire ("DC")
                FinSi
        Sinon
                Si(y > 0) Alors Ecrire ("AD")
                SinonSi (y < 0) Alors Ecrire ("BC")
                Sinon Ecrire ("ABCD")
                FinSi
        FinSi
Fin
```

Python

```
# Lecture des coordonnées du point
x = float(input("Saisir la coordonnée x du point : "))
y = float(input("Saisir la coordonnée y du point : "))

# Détermination de la partie du plan
if x > 0:
    if y > 0:
        print("A")  # Partie A : x > 0, y > 0
    elif y < 0:
        print("B")  # Partie B : x > 0, y < 0
    else:
        print("AB")  # Sur l'axe entre A et B : x > 0, y = 0
elif x < 0:
    if y > 0:
        print("D")  # Partie D : x < 0, y > 0
    elif y < 0:
        print("C")  # Partie C : x < 0, y < 0
    else:
        print("DC")  # Sur l'axe entre D et C : x < 0, y = 0
else:
    if y > 0:
        print("AD")  # Sur l'axe entre A et D : x = 0, y > 0
    elif y < 0:
        print("BC")  # Sur l'axe entre B et C : x = 0, y < 0
    else:
```

```
print("ABCD")  # Origine (0, 0)
```

Explications du code :

- **Quadrants** :
  - **A** : x>0x > 0x>0 et y>0y > 0y>0
  - **B** : x>0x > 0x>0 et y<0y < 0y<0
  - **C** : x<0x < 0x<0 et y<0y < 0y<0
  - **D** : x<0x < 0x<0 et y>0y > 0y>0
- **Axes** :
  - **AB** : x>0x > 0x>0 et y=0y = 0y=0 (axe horizontal à droite de l'origine)
  - **DC** : x<0x < 0x<0 et y=0y = 0y=0 (axe horizontal à gauche de l'origine)
  - **AD** : x=0x = 0x=0 et y>0y > 0y>0 (axe vertical au-dessus de l'origine)
  - **BC** : x=0x = 0x=0 et y<0y < 0y<0 (axe vertical en dessous de l'origine)
- **Origine** :
  - Si x=0x = 0x=0 et y=0y = 0y=0, le point est à l'origine, donc on affiche **"ABCD"**.

UNIVERSITE IBN ZOHR
FACULTE DES SCIENCES
AGADIR

جامعة ابن زهر
كليـــة العلـــوم
أكاديـر

Algorithmique et Programmation (SMP3)
TD2 : Instructions élémentaires
2024-2025

Exercice5
Ecrire un algorithme qui lit trois valeurs entières (A, B et C) et qui permet de les trier par échanges successifs. Enfin, il les affiche dans l'ordre.

```
Algorithme TriSuccessif
Variables A, B, C, Z : Entier

Debut
 Ecrire (" entrer Les valeurs de A, B et C") Lire (A, B, C)
Si (A > B) ALORS
Z ← A
A ← B
B ← Z
 FinSi
Si (B > C) ALORS
Z ← B
B ← C
C ← Z
Si (A > B) ALORS
Z ← A
A ← B
B ← Z
FinSi
FinSi
Ecrire ("Les valeurs A , B et C sont (dans l'ordre):", A , " ", B, " ", C)
Fin
```

Python

```
# Demander les trois valeurs à l'utilisateur
A = int(input("Entrez la valeur de A : "))
B = int(input("Entrez la valeur de B : "))
C = int(input("Entrez la valeur de C : "))

# Premier échange entre A et B si nécessaire
if A > B:
    A, B = B, A

# Deuxième échange entre B et C si nécessaire
if B > C:
    B, C = C, B

    # Vérifier de nouveau A et B après l'échange
    if A > B:
        A, B = B, A

# Afficher les valeurs triées
print("Les valeurs A, B et C dans l'ordre croissant sont :", A, B, C)
```

Exercice 6
Ecrire un algorithme permettant de résoudre une équation du second degré.

Ecrire un algorithme permettant de résoudre une équation du second degré.

```
Algorithme SecondDegre
Variables a, b, c, delta : Reel
Debut
        Ecrire (" saisissez les valeurs a, b et c de l'équation ax²+bx+c=0 : ")
        Lire (a, b, c)
        Si (a=0) Alors
                Ecrire (" équation du premier degré ")
                Si (b <> 0) ALORS
                        Ecrire ("La solution est ", -c/b)
                SinonSi (c = 0) Alors
                        Ecrire (" Tous les nombres vérifient l'équation ")
                Sinon  Ecrire (" Pas de solution")
                FinSi
        Sinon
                delta ← b*b-4*a*c
                Si (delta > 0) Alors
                        Ecrire ("La première solution est :  ", (-b-sqrt(delta ))/(2*a) )
                        Ecrire ("La deuxième solution est : ", (-b+sqrt(delta ))/(2*a) )
                SinonSi (delta =0) Alors
                        Ecrire ( "La solution est : ", -b/(2a))
                Sinon Ecrire ("pas de solutions réelles !!")
                FinSi
        FinSi
Fin
```

Python

```
import math

# Demander les coefficients a, b et c
a = float(input("Saisissez la valeur de a : "))
b = float(input("Saisissez la valeur de b : "))
c = float(input("Saisissez la valeur de c : "))

# Cas d'une équation du premier degré si a = 0
if a == 0:
    print("Équation du premier degré.")
    if b != 0:
        print("La solution est :", -c / b)
```

```
    elif c == 0:
        print("Tous les nombres vérifient l'équation.")
    else:
        print("Pas de solution.")
else:
    # Calcul du discriminant (delta)
    delta = b ** 2 - 4 * a * c

    # Cas où delta est positif : deux solutions réelles distinctes
    if delta > 0:
        x1 = (-b - math.sqrt(delta)) / (2 * a)
        x2 = (-b + math.sqrt(delta)) / (2 * a)
        print("La première solution est :", x1)
        print("La deuxième solution est :", x2)

    # Cas où delta est nul : une solution réelle unique
    elif delta == 0:
        x = -b / (2 * a)
        print("La solution est :", x)

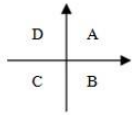
    # Cas où delta est négatif : pas de solutions réelles
    else:
        print("Pas de solutions réelles !!")
```

Exemples d'utilisation :

- Pour l'équation x2−3x+2=0x^2 - 3x + 2 = 0x2−3x+2=0, entrez `a = 1`, `b = -3`, `c = 2`. Les solutions sont 1 et 2.
- Pour une équation du premier degré 0×x2+2x−4=00 \times x^2 + 2x - 4 = 00×x2+2x−4=0, entrez `a = 0`, `b = 2`, `c = -4`. La solution est 2.

Exercice 7

Écrire un algorithme qui, étant donné les coordonnées x et y d'un point, détermine dans quelle partie (A, B, C ou D) du plan se trouve le point (cf. figure1).



Algorithme PointPlan
Variables x, y : Reel
Debut
Ecrire ("Saisir les coordonnées du point")
Lire (x, y)
Si (x > 0) Alors
Si(y > 0) Alors Ecrire ("A")
SinonSi (y < 0) Alors Ecrire ("B")
Sinon Ecrire ("AB")
FinSi
SinonSi (x < 0) Alors
Si(y > 0) Alors Ecrire ("D")
SinonSi (y < 0) Alors Ecrire ("C")
Sinon Ecrire ("DC")
FinSi
Sinon
Si(y > 0) Alors Ecrire ("AD")
SinonSi (y < 0) Alors Ecrire ("BC")
Sinon Ecrire ("ABCD")
FinSi
FinSi
Fin

Python

```
# Lecture des coordonnées du point
x = float(input("Saisir la coordonnée x du point : "))
y = float(input("Saisir la coordonnée y du point : "))

# Détermination de la partie du plan
if x > 0:
    if y > 0:
        print("A")  # Partie A : x > 0, y > 0
    elif y < 0:
        print("B")  # Partie B : x > 0, y < 0
    else:
        print("AB")  # Sur l'axe entre A et B : x > 0, y = 0
elif x < 0:
    if y > 0:
        print("D")  # Partie D : x < 0, y > 0
    elif y < 0:
        print("C")  # Partie C : x < 0, y < 0
    else:
        print("DC")  # Sur l'axe entre D et C : x < 0, y = 0
else:
    if y > 0:
        print("AD")  # Sur l'axe entre A et D : x = 0, y > 0
    elif y < 0:
        print("BC")  # Sur l'axe entre B et C : x = 0, y < 0
    else:
```

```
print("ABCD")  # Origine (0, 0)
```

Explications du code :

- **Quadrants** :
  - **A** : x>0x > 0x>0 et y>0y > 0y>0
  - **B** : x>0x > 0x>0 et y<0y < 0y<0
  - **C** : x<0x < 0x<0 et y<0y < 0y<0
  - **D** : x<0x < 0x<0 et y>0y > 0y>0
- **Axes** :
  - **AB** : x>0x > 0x>0 et y=0y = 0y=0 (axe horizontal à droite de l'origine)
  - **DC** : x<0x < 0x<0 et y=0y = 0y=0 (axe horizontal à gauche de l'origine)
  - **AD** : x=0x = 0x=0 et y>0y > 0y>0 (axe vertical au-dessus de l'origine)
  - **BC** : x=0x = 0x=0 et y<0y < 0y<0 (axe vertical en dessous de l'origine)
- **Origine** :
  - Si x=0x = 0x=0 et y=0y = 0y=0, le point est à l'origine, donc on affiche **"ABCD"**.

Exercice 8

Ecrire un algorithme permettant de lire un caractère au clavier et de déterminer s'il s'agit d'une lettre majuscule, minuscule, un chiffre ou un caractère spécial. Ensuite, rédige le code en Python correspondant.

```
Algorithme lecture_Caractere
Variable C : Caractere
Début
        Ecrire ("taper un caractère au clavier")
        Lire (C)
        Selon (C)
                cas 'A'…'Z' : Ecrire (C, " est une majuscule")
                cas 'a'…'z' : Ecrire (C, " est une minuscule")
                cas '0'…'9' : Ecrire (C, " est un chiffre")
                Sinon : Ecrire (C, " est un caractère spécial")
        FinSelon
Fin
```

```
# Lecture du caractère

caractere = input("Veuillez saisir un caractère : ")

# Vérification du type de caractère

if 'A' <= caractere <= 'Z':

    print("Il s'agit d'une lettre majuscule.")

elif 'a' <= caractere <= 'z':

    print("Il s'agit d'une lettre minuscule.")

elif '0' <= caractere <= '9':

    print("Il s'agit d'un chiffre.")

else:

    print("Il s'agit d'un caractère spécial.")
```

Exercice 9

Ecrire un algorithme calculatrice permettant la saisie du premier entier (a) de l'opération (+ ou – ou * ou / : sont des caractères) et du deuxième entier (b) et qui affiche le résultat.

```
Algorithme Calculatrice
Variables a, b : Entier
          Op :Caractere
Debut
        Ecrire("saisir le premier entier")
        Lire (a)
        Ecrire("saisir l'opération")
        Lire (Op)
        Ecrire("saisir le deuxième entier non nul")
        Lire (b)
        Selon Op
                Cas '+' : Ecrire ("la somme de ", a, " et ", b, " est égale à ", a+b)
                Cas '-' : Ecrire ("le résultat de la soustraction de ", a, " et ", b, " est : ", a-b)
                Cas '*' : Ecrire ("le résultat de la multiplication de ", a, " et ", b, "est : ", a*b)
                Cas '/' : Ecrire ("le résultat de la division de ", a, " et ", b, " est: ", a/b)
                Sinon : Ecrire ("opération non valide")
        FinSelon
fin
```

```
# Saisie des valeurs

a = int(input("Saisir le premier entier : "))

op = input("Saisir l'opération (+, -, *, /) : ")
```

```
b = int(input("Saisir le deuxième entier : "))


# Exécution de l'opération selon le choix de l'utilisateur
if op == '+':
    print("La somme de", a, "et", b, "est égale à", a + b)
elif op == '-':
    print("Le résultat de la soustraction de", a, "et", b, "est :", a - b)
elif op == '*':
    print("Le résultat de la multiplication de", a, "et", b, "est :", a * b)
elif op == '/':
    if b != 0:
        print("Le résultat de la division de", a, "et", b, "est :", a / b)
    else:
        print("Erreur : Division par zéro")
else:
    print("Opération non valide")
```

Exercice 10

Écrire un algorithme qui demande un numéro de mois à l'utilisateur et indique en retour son nom et le nombre de jours dans ce mois.

```
Algorithme Mois
Variable N : entier
Début
    Ecrire ("donner un numéro de mois")
    Lire (N)
    Selon N
        Cas 1 : Ecrire ("janvier 31 jours")
        Cas 2 : Ecrire ("Février 28 ou 29 jours")
        Cas 3 : Ecrire ("Mars 31 jours")
        Cas 4 : Ecrire ("Avril 30 jours")
        Cas 5 : Ecrire ("Mai 31 jours")
        Cas 6 : Ecrire ("Juin 30 jours")
        Cas 7 : Ecrire ("Juillet 31 jours")
        Cas 8 : Ecrire ("Aout 31 jours")
        Cas 9 : Ecrire ("Septembre 30 jours")
        Cas 10 : Ecrire ("Octobre 31 jours")
        Cas 11 : Ecrire ("Novembre 30 jours")
        Cas 12 : Ecrire ("Décembre 31 jours")
        Sinon : Ecrire(" numéro de mois invalide")
    FinSelon
Fin
```

```
# Saisie du numéro de mois
N = int(input("Donner un numéro de mois : "))


# Vérification du numéro de mois et affichage du résultat
if N == 1:
    print("Janvier 31 jours")
elif N == 2:
    print("Février 28 ou 29 jours")
elif N == 3:
    print("Mars 31 jours")
elif N == 4:
```

```
    print("Avril 30 jours")
elif N == 5:
    print("Mai 31 jours")
elif N == 6:
    print("Juin 30 jours")
elif N == 7:
    print("Juillet 31 jours")
elif N == 8:
    print("Août 31 jours")
elif N == 9:
    print("Septembre 30 jours")
elif N == 10:
    print("Octobre 31 jours")
elif N == 11:
    print("Novembre 30 jours")
elif N == 12:
    print("Décembre 31 jours")
else:
    print("Numéro de mois invalide")
```

Exercice 1

1. Ecrire un algorithme qui demande un nombre de départ, et qui calcule la somme des entiers jusqu'à ce nombre.

```
Algorithme somme_entier
Variables i, N, S : entier
Début
        Ecrire (" donner un entier ")
        Lire (N)
        S ← 0
        Pour i = 1 à N
                S ← S + i
        FinPour
        Ecrire ("la somme est :", S)
Fin
```

```
# Saisie du nombre de départ
N = int(input("Entrez un nombre de départ : "))

# Initialisation de la somme
somme = 0

# Boucle pour additionner chaque entier de 1 jusqu'à N
for i in range(1, N + 1):
    somme += i

# Affichage du résultat
print("La somme des entiers jusqu'à", N, "est :", somme)
```

2. Ecrire un algorithme qui affiche la somme des valeurs absolues des entiers pairs compris entre deux entiers relatifs a et b saisis par l'utilisateur.

```
Algorithme somme_Absolues
Variables i, S, a, b : entier
Début
        Ecrire (" Saisir deux entiers dans l'ordre croissant ")
        Lire (a, b)
        S ← 0
        Pour i = a A b
                Si (i mod 2= 0) Alors S ← S + abs ( i )
                FinSi
        FinPour
        Ecrire ("la somme est :", S)
Fin
```

```
# Saisie des deux entiers

a = int(input("Entrez le premier entier (a) : "))

b = int(input("Entrez le deuxième entier (b) : "))



# Initialisation de la somme

somme = 0



# Détermination de l'intervalle en s'assurant que a <= b

for i in range(min(a, b), max(a, b) + 1):

    # Vérification si le nombre est pair

    if i % 2 == 0:

        # Ajout de la valeur absolue du nombre pair à la somme

        somme += abs(i)



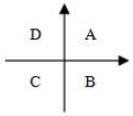
# Affichage du résultat

print("La somme des valeurs absolues des entiers pairs entre", a, "et", b, "est :", somme)
```

Algorithmique et Programmation (SMP3)
TD3 : Instructions élémentaires
2024-2025

Exercice 7

Écrire un algorithme qui, étant donné les coordonnées x et y d'un point, détermine dans quelle partie (A, B, C ou D) du plan se trouve le point (cf. figure1).




```
Algorithme PointPlan
Variables x, y : Reel
Debut
        Ecrire ("Saisir les coordonnées du point")
        Lire (x, y)
        Si (x > 0) Alors
                Si(y > 0) Alors Ecrire ("A")
                SinonSi (y < 0) Alors Ecrire ("B")
                Sinon Ecrire ("AB")
                FinSi
        SinonSi (x < 0) Alors
                Si(y > 0) Alors Ecrire ("D")
                SinonSi (y < 0) Alors Ecrire ("C")
                Sinon Ecrire ("DC")
                FinSi
        Sinon
                Si(y > 0) Alors Ecrire ("AD")
                SinonSi (y < 0) Alors Ecrire ("BC")
                Sinon Ecrire ("ABCD")
                FinSi
        FinSi
    Fin
```

Python

```
# Lecture des coordonnées du point
x = float(input("Saisir la coordonnée x du point : "))
y = float(input("Saisir la coordonnée y du point : "))

# Détermination de la partie du plan
if x > 0:
   if y > 0:
      print("A")  # Partie A : x > 0, y > 0
   elif y < 0:
      print("B")  # Partie B : x > 0, y < 0
   else:
      print("AB")  # Sur l'axe entre A et B : x > 0, y = 0
elif x < 0:
   if y > 0:
      print("D")  # Partie D : x < 0, y > 0
   elif y < 0:
      print("C")  # Partie C : x < 0, y < 0
   else:
      print("DC")  # Sur l'axe entre D et C : x < 0, y = 0
else:
   if y > 0:
      print("AD")  # Sur l'axe entre A et D : x = 0, y > 0
   elif y < 0:
      print("BC")  # Sur l'axe entre B et C : x = 0, y < 0
   else:
      print("ABCD")  # Origine (0, 0)
```

Exercice 1

1. Ecrire un algorithme qui demande un nombre de départ, et qui calcule la somme des entiers jusqu'à ce nombre.

```
Algorithme somme_entier
Variables i, N, S : entier
Début
        Ecrire (" donner un entier ")
        Lire (N)
        S ← 0
        Pour i = 1 à N
                S ← S + i
        FinPour
        Ecrire ("la somme est :", S)
Fin
```

```
# Saisie du nombre de départ

N = int(input("Entrez un nombre de départ : "))

# Initialisation de la somme

somme = 0

# Boucle pour additionner chaque entier de 1 jusqu'à N

for i in range(1, N + 1):
```

```
    somme += i

# Affichage du résultat

print("La somme des entiers jusqu'à", N, "est :", somme)
```

2. Ecrire un algorithme qui affiche la somme des valeurs absolues des entiers pairs compris entre deux entiers relatifs a et b saisis par l'utilisateur.

```
Algorithme somme_Absolues
Variables i, S, a, b : entier
Début
        Ecrire (" Saisir deux entiers dans l'ordre croissant ")
        Lire (a, b)
        S ← 0
        Pour i = a A b
                Si (i mod 2= 0) Alors S ← S + abs ( i )
                FinSi
        FinPour
        Ecrire ("la somme est :", S)
Fin
```

```
# Saisie des deux entiers

a = int(input("Entrez le premier entier (a) : "))

b = int(input("Entrez le deuxième entier (b) : "))

# Initialisation de la somme

somme = 0

# Détermination de l'intervalle en s'assurant que a <= b

for i in range(min(a, b), max(a, b) + 1):

    # Vérification si le nombre est pair

    if i % 2 == 0:

        # Ajout de la valeur absolue du nombre pair à la somme

        somme += abs(i)

# Affichage du résultat

print("La somme des valeurs absolues des entiers pairs entre", a, "et", b, "est :", somme)
```

Exercice 2

Ecrire un algorithme qui demande un nombre de départ, et qui ensuite affiche la table de multiplication de ce nombre.

```
Algorithme table_multiplication
Variables i, x : entier
Début
        Ecrire ("donner un entier")
        Lire (x)
        Pour i = 1 A 10
                Ecrire (i, " * ",  x, " = ", i*x)
        FinPour
Fin
```

```
# Saisie du nombre de départ

N = int(input("Entrez un nombre de départ : "))

# Boucle pour afficher la table de multiplication de 1 à 10

print(f"Table de multiplication de {N}:")

for i in range(1, 11):

    # Calcul du produit

    resultat = N * i

    # Affichage du résultat

    print(f"{N} x {i} = {resultat}")
```

Exercice 3

1. Ecrire un algorithme qui lit un entier positif N et qui affiche la somme des N premières puissances de 2. ($S= 2^0 + 2^1 + 2^2 + \dots + 2^N$)

```
Algorithme somme_Puissances
Variables i, N, S : entier
Début
        Ecrire (" Saisir un entier ")
        Lire (N)
        S ← 0
        Pour i = 0 à N
                S ← S + 2 ^ i
        FinPour
        Ecrire ("la somme est : ", S)
Fin
```

```
# Saisie de l'entier positif N

N = int(input("Entrez un entier positif N : "))

# Initialisation de la somme

somme = 0

# Boucle pour calculer la somme des puissances de 2, de 2^0 à 2^N

for i in range(N + 1):

    somme += 2 ** i

# Affichage du résultat

print("La somme des", N, "premières puissances de 2 est :", somme)
```

2. Ecrire un algorithme qui calcule la somme des inverses des carrés des n premiers entiers, n étant donné par l'utilisateur. ($S= 1/1^2 + 1/2^2 + \dots + 1/n^2$)

```
Algorithme somme_InverseCarre
Variables i, n: entier
        S : Reel
Début
        Ecrire (" Saisir un entier ")
        Lire (n)
        S ← 0
        Pour i = 1 à n
                S ← S + 1 / (i ^ 2)
        FinPour
        Ecrire ("la somme est : ", S)
Fin
```

```
# Saisie de l'entier positif n

n = int(input("Entrez un entier positif n : "))

# Initialisation de la somme

somme = 0.0

# Boucle pour calculer la somme des inverses des carrés

for i in range(1, n + 1):

    somme += 1 / (i ** 2)

# Affichage du résultat

print("La somme des inverses des carrés des", n, "premiers entiers est :", somme)
```

3. Ecrire un algorithme qui imprime la suite des N puissances de x (nombre réel donné) : x, x^2, . . ., x^N (N étant donné par l'utilisateur).

```
Algorithme PuissanceX
Variables i, N: entier
        X : Reel
Début
        Ecrire (" Saisir un entier ")
        Lire (N)
        Pour i = 1 à N
                Ecrire (X ^ i, "  ")
        FinPour
Fin
```

```
# Saisie des valeurs de x et N

x = float(input("Entrez un nombre réel x : "))

N = int(input("Entrez un entier positif N : "))

# Affichage des N puissances de x

print("Les", N, "premières puissances de", x, "sont :")

for i in range(1, N + 1):

    puissance = x ** i
```

```
print(f"x^{i} = {puissance}")
```

1. Écrire un algorithme qui imprime la suite des carrés et des cubes des entiers compris entre 10 et 100.

```
Algorithme CarreCube
Variables i, carre, cube: entier
Début
        Pour i = 10 à 100
                carre ← i × i
                cube ← carre × i
                ecrire (carre, " ", cube, "\n")
        FinPour
Fin
```

```
# Parcours des entiers de 10 à 100

print("Nombre, Carré, Cube")

for n in range(10, 101):

    carre = n ** 2

    cube = n ** 3

    print(f"Nombre : {n}, Carré : {carre}, Cube : {cube}")
```

5. Ecrire un algorithme qui imprime les termes de la suite alternée :
   $1, -2!, 3!, -4!, \ldots, (-1)^{(n+1)} \times n!$

```
Algorithme ImprimerSuite
Variables i, n, F: entier
Début
        Ecrire (" Saisir un entier ")
        Lire (n)
        F ← -1
        Pour i = 1 à n
                F ← (-1) * F * i
                Ecrire (F, " ")
        FinPour
Fin
```

```
import math

# Saisie du nombre de termes souhaité

n = int(input("Entrez un entier positif pour le nombre de termes : "))

# Génération et affichage de la suite alternée

for i in range(1, n + 1):

    signe = (-1) ** (i + 1)  # Signe alterné

    terme = signe * math.factorial(i)  # Terme avec factorielle et signe alterné

    print(terme, end=", " if i < n else "\n")  # Affiche le terme avec des virgules entre les termes
```

2 solution

```
n = int(input("Saisir un entier : "))
F = -1

for i in range(1, n + 1):
    F = (-1) * F * i
    print(F, end=", ")
```

Exercice 4

Ecrire un algorithme qui demande à l'utilisateur un nombre compris entre 0 et 10 (bornes incluses) jusqu'à ce que la réponse convienne.

```
Algorithme ControleSaisie
Variables N: entier
Début
        Ecrire (" donner un entier ")
        Lire (N)
        TantQue (N <0 OU N>10)
                Ecrire ("Fausse réponse !!! réessayez !! ")
                Lire (N)
        FinTantQue
        Ecrire ("réponse correcte")
Fin
```

```
# Demande initiale de saisie
```

```
N = int(input("Donner un entier entre 0 et 10 : "))
# Boucle de contrôle de la saisie
while N < 0 or N > 10:
    print("Fausse réponse !!! réessayez !!")
    N = int(input("Donner un entier entre 0 et 10 : "))
# Message de validation
print("Réponse correcte")
```

Exercice 5

Ecrire un algorithme qui affiche la somme des prix d'une suite d'articles en DH (entiers) saisis par l'utilisateur et se terminant par zéro.

Algorithme Somme_Prix
Variables prix, S : Entier

Debut
S ← 0
Ecrire("Entrer le prix du 1° article:")
Lire (prix)
TantQue (prix <> 0)
S ← S + prix
Ecrire("Entrer le prix de l'article suivant(0 si Fin):")
Lire(prix)
FinTantQue
Ecrire (" La somme des prix des articles est ", S)
Fin

```
# Initialisation de la somme
S = 0
# Saisie initiale du prix
prix = int(input("Entrer le prix du 1° article (0 pour terminer) : "))
# Boucle pour accumuler la somme des prix
while prix != 0:
    S += prix
    prix= int(input("Entrer le prix de l'article suivant (0 pour terminer) : "))
# Affichage de la somme totale
print("La somme des prix des articles est :", S, "DH")
```

Algorithmique et Programmation (SMP3)
TD3 : Instructions élémentaires
2024-2025

Exercice 4

Ecrire un algorithme qui demande à l'utilisateur un nombre compris entre 0 et 10 (bornes incluses) jusqu'à ce que la réponse convienne.

```
Algorithme ControleSaisie
Variables N: entier
Début
        Ecrire (" donner un entier ")
        Lire (N)
        TantQue (N <0 OU N>10)
                Ecrire ("Fausse réponse !!! réessayez !! ")
                Lire (N)
        FinTantQue
        Ecrire ("réponse correcte")
Fin
```

```
# Demande initiale de saisie

N = int(input("Donner un entier entre 0 et 10 : "))

# Boucle de contrôle de la saisie

while N < 0 or N > 10:

    print("Fausse réponse !!! réessayez !!")

    N = int(input("Donner un entier entre 0 et 10 : "))

# Message de validation

print("Réponse correcte")
```

Exercice 5

Ecrire un algorithme qui affiche la somme des prix d'une suite d'articles en DH (entiers) saisis par l'utilisateur et se terminant par zéro.

```
Algorithme Somme_Prix
Variables prix, S : Entier
```



```
Debut
        S ← 0
        Ecrire("Entrer le prix du 1° article:")
        Lire (prix)
        TantQue (prix <> 0)
                S ← S + prix
                Ecrire("Entrer le prix de l'article suivant( 0 si Fin):")
                Lire(prix)
        FinTantQue
        Ecrire (" La somme des prix des articles est ", S)
Fin
```

```
# Initialisation de la somme

S = 0

# Saisie initiale du prix

prix = int(input("Entrer le prix du 1° article (0 pour terminer) : "))


# Boucle pour accumuler la somme des prix

while prix != 0:
```

```
    S += prix

    prix= int(input("Entrer le prix de l'article suivant (0 pour terminer) : "))

# Affichage de la somme totale

print("La somme des prix des articles est :", S, "DH")
```

Exercice 6

Ecrire un algorithme permettant la saisie d'une suite d'entiers se terminant par zéro et vérifier si cette suite contient deux entiers consécutifs égaux.

```
Algorithme NombreConsecutifs
Variables a, b, t : Entier
Debut
        t ← 0
        Ecrire("Entrer un nombre ")
        Lire (a)
        TantQue (a <> 0)
                b ← a
                Ecrire("Entrer le nombre suivant( 0 si Fin):")
                Lire(a)
                Si (a = b) Alors t ← 1
        FinTantQue
        Si (t =1) Alors Ecrire (" La suite contient deux entiers consécutifs égaux ")
        Sinon Ecrire (" La suite ne contient  pas deux entiers consécutifs égaux ")
        FinSi
Fin
```

```
# Initialisation

t = 0

# Saisie du premier nombre

a = int(input("Entrer un nombre (0 pour terminer) : "))

while a != 0:

    b = a

    # Saisie du nombre suivant

    a = int(input("Entrer le nombre suivant (0 pour terminer) : "))

    # Vérification des nombres consécutifs

    if a == b:

        t = 1

# Résultat final

if t == 1:

    print("La suite contient deux entiers consécutifs égaux.")

else:

    print("La suite ne contient pas deux entiers consécutifs égaux.")
```

Exercice 7

Ecrire un algorithme qui affiche le maximum d'une suite se terminant par -1. (-1 n'est pas pris en compte dans le calcul du max)

```
Algorithme MaxSuite
Variables a, max: Entier
Debut
        Ecrire("Entrer un nombre ")
        Lire (a)
        max ← a
        TantQue (a <> -1)
                Ecrire("Entrer le nombre suivant( -1 si Fin):")
                Lire(a)
                Si (max < a Et a <> -1) Alors max ← a
                FinSi
        FinTantQue
        Ecrire (" Le maximum de la suite saisie est :", max)
Fin
```

```
# Initialisation

a = int(input("Entrer un nombre: "))

maximum = a

# Boucle pour lire la suite

while a != -1:

    a = int(input("Entrer le nombre suivant (-1 si Fin): "))

    if a != -1 and a > maximum:

        maximum = a
```

```
# Affichage du maximum

print("Le maximum de la suite saisie est :", maximum)
```

Exercice 8

Ecrire un algorithme mettant en œuvre le jeu suivant : Le premier utilisateur saisit un entier que le second doit deviner. Pour cela, il a le droit à autant de tentatives qu'il souhaite. A chaque échec, le programme lui indique si l'entier cherché est plus grand ou plus petit que sa proposition. Un score indiquant le nombre de coups joués est mis à jour et affiché lorsque l'entier est trouvé.

```
Algorithme devinette
Variables a, n, t : entier
Debut
        Ecrire(" entrez le nombre à deviner")
        Lire (a)
        Ecrire("entrez le nombre : premier essai" )
        Lire (n)
        t ← 1
        TantQue ( a<>n )
                Si (n > a) Alors
                        Ecrire (" nombre cherché plus petit que : ", n)
                Sinon
                        Ecrire (" nombre cherché plus grand que ", n)
                FinSi
                t ← t+1
                Ecrire("entrez un autre nombre : tentative n° ", t)
                Lire (n)
        FinTantQue
        Ecrire (" c'est gagné : le nombre de tentatives est", t)
Fin
```

```
# Lecture du nombre à deviner

a = int(input("Entrez le nombre à deviner : "))

# Initialisation des variables

n = int(input("Entrez le nombre : premier essai : "))

t = 1  # Nombre de tentatives

# Boucle pour continuer jusqu'à ce que le nombre soit deviné

while n != a:

    if n > a:

        print(f"Nombre cherché plus petit que : {n}")

    else:

        print(f"Nombre cherché plus grand que : {n}")

        # Mise à jour du nombre de tentatives et nouvelle entrée

    t += 1

    n = int(input(f"Entrez un autre nombre : tentative n° {t} : "))

# Affichage du résultat

print(f"C'est gagné : le nombre de tentatives est {t}")
```

Exercice 9

1. Écrire un algorithme qui détermine si un nombre saisi par l'utilisateur est premier ou non.

```
Algorithme Premier
Variables i, N, compt : entier
Début
        Ecrire (" donner un entier ")
        Lire (N)
        compt ← 0
        Pour i = 1 à N
                Si (N mod i = 0) Alors compt ← compt + 1
        FinPour
        Si (compt = 2) Alors Ecrire (" Nombre premier ")
        Sinon Ecrire (" Nombre non premier ")
        FinSi
Fin
```

```
# Lecture du nombre

N = int(input("Donner un entier : "))

# Initialisation du compteur

compt = 0

# Boucle pour vérifier les diviseurs

for i in range(1, N + 1):

    if N % i == 0:

        compt += 1
```

```
# Vérification du nombre de diviseurs

if compt == 2:

    print("Nombre premier")

else:

    print("Nombre non premier")
```

2. Écrire un algorithme qui affiche tous les nombres premiers inférieurs à 100

```
Algorithme
PremierInferieur100
Variables i, N, compt :
entier Début
    Pour N= 1 A 100
        comp ← 0
        Pour i = 1 à N
            Si (N mod i = 0) Alors compt ← compt + 1
        FinPour
        Si (compt = 2) Alors Ecrire (N, " ")
        FinSi
    FinPour
Fin
```

```
print("Nombres premiers inférieurs à 100 :")

for N in range(2, 101):  # Parcourir les nombres de 2 à 100

    compt = 0

    for i in range(1, N + 1):  # Vérifier les diviseurs

        if N % i == 0:

            compt += 1

    if compt == 2:  # Si exactement deux diviseurs, le nombre est premier

        print(N, end=" ")
```

Exercice 10

Ecrire un algorithme qui imprime pour n donné :

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
.....
.....
1 2 3 4 5 6 ... n
```

```
Algorithme affichage_ligne
Variables n, i, j : entier
Début
    Pour i = 1 A n
        Pour j = 1 à i
            Ecrire (j, " ")
        FinPour
        Ecrire ("\n")
    FinPour
Fin
```

```
# Lire la valeur de n

n = int(input("Donner la valeur de n : "))

# Boucle pour afficher les lignes

for i in range(1, n + 1):  # De 1 à n inclus

    for j in range(1, i + 1):  # De 1 à i inclus

        print(j, end=" ")  # Afficher les nombres sur une seule ligne

    print()  # Passer à la ligne suivante après chaque ligne
```

Exercice 11

Concevoir un algorithme qui, pour un caractère imprimable et un nombre n donnés,

1. Imprime une barre horizontale de n de ce caractère.

```
Algorithme Barre_horizontale
Variables n , i : entier
        Car : Caractere
Debut

Ecrire ("donner le caractère à imprimer") Lire
(car)
Ecrire ("Saisir la longueur de la barre") Lire (n)
Pour i = 1 A n
Fin
        Ecrire (car)
FinPour
```

```
# Lire le caractère et la longueur de la barre

car = input("Donner le caractère à imprimer : ")

n = int(input("Saisir la longueur de la barre : "))

# Affichage de la barre horizontale

for i in range(1, n + 1):  # De 1 à n inclus

    print(car, end="")  # Affiche le caractère sur une seule ligne

print()  # Passer à la ligne suivante après l'affichage
```

2. Modifier l'algorithme pour l'impression d'une barre d'épaisseur quelconque donnée

```
Algorithme barre_horizontale
Variables n , p, i, j: entier
        Car : caractère
Début
        Ecrire ("donner le caractère à imprimer")
        Lire (car)
        Ecrire ("Saisir la longueur de la barre")
        Lire (n)
        Ecrire ("Saisir l'épaisseur de la barre")
        Lire (p)
        Pour j = 1 A p
                Pour i = 1 A n
                        Ecrire (car)
                FinPour
                Ecrire ("\n")
        FinPour
Fin
```

```
# Lire le caractère, la longueur et l'épaisseur de la barre

car = input("Donner le caractère à imprimer : ")

n = int(input("Saisir la longueur de la barre : "))

p = int(input("Saisir l'épaisseur de la barre : "))

# Affichage de la barre avec l'épaisseur

for j in range(p):  # Boucle pour gérer l'épaisseur

    for i in range(n):  # Boucle pour gérer la longueur

        print(car, end="")  # Affiche le caractère sur une ligne

    print()  # Passe à la ligne après chaque bande
```

Exercice 12

Ecri e un algorithme qui demande l'heure actuelle (heures, minutes et secondes) et prédit l'heure qui sera dans 30 s.

Algorithme Predire_heure

Variables hh1, mm1, ss1, hh2, mm2n ss2 : Entier

Début

Répéter

Ecrire ("Entrer l'heure: ") Lire (hh1)

Jusqu'à (hh1 >= 0 ET hh1 <= 23) Répéter

Ecrire ("Entrer les minutes : ") Lire (mm1)

Jusqu'à (mm1>= 0 et mm1 <= 59) Répéter

Ecrire ("Entrer les secondes : ") Lire (ss1)

Jusqu'à (ss1 >= 0 et ss1 <= 59) ss2 ← ss1 + 30

```
mm2 ← mm1
hh2 ← hh1
Si (ss2 > 59) Alors
ss2 ← ss2 – 60 mm2 ← mm2 + 1
Si (mm2 > 59) Alors
mm2 ← mm2 – 60
hh2 ← hh2 + 1
Si ( hh2 > 23 ) Alors
hh2 ← hh2 – 24
FinSi
FinSi
FinSi
Fin
Ecrire ("Il est : ", hh1," : ", mm1," : ", ss1)
Ecrire ("Dans 30s, il sera : ", hh2, " : ", mm2, " : ", ss2)
```

```
# Fonction pour prédire l'heure dans 30 secondes
def predire_heure():
    # Saisie de l'heure actuelle
    while True:
        hh1 = int(input("Entrer l’heure (0-23) : "))
        if 0 <= hh1 <= 23:
            break
while True:
    mm1 = int(input("Entrer les minutes (0-59) : "))
    if 0 <= mm1 <= 59:
        break
while True:
    ss1 = int(input("Entrer les secondes (0-59) : "))
    if 0 <= ss1 <= 59:
        break
# Calcul de l'heure dans 30 secondes
ss2 = ss1 + 30
mm2 = mm1
hh2 = hh1
# Gestion des dépassements
if ss2 > 59:
    ss2 -= 60
    mm2 += 1
if mm2 > 59:
    mm2 -= 60
    hh2 += 1
if hh2 > 23:
    hh2 -= 24
# Affichage du résultat
```

```
    print(f"L'heure dans 30 secondes sera : {hh2:02}:{mm2:02}:{ss2:02}")


# Appel de la fonction

predire_heure()
```

Pourquoi utiliser `break` ici ?

1. **Validation des entrées utilisateur :**
   - Les boucles `while True` permettent de répéter la demande tant que l'entrée n'est pas correcte.
   - Une fois une valeur correcte entrée (par exemple, une heure entre 0 et 23), le `break` arrête la boucle.
2. **Avantage :**
   - Cela garantit que le programme ne continue pas tant que l'utilisateur n'a pas saisi des valeurs valides.

UNIVERSITE IBN ZOHR
FACULTE DES SCIENCES
AGADIR

جامعة ابن زهر
كليـــة العلـــوم
أكاديـر

Pr. Lachhab Fadwa

Algorithmique et Programmation (SMP3)
TD4 : Instructions élémentaires
2024-2025

Exercice 1

Soit T un tableau de N notes saisies par l'utilisateur. Ecrire l'algorithme qui permet de :
1. Calculer la somme des éléments de ce tableau, ainsi que leur moyenne.
2. Afficher les notes qui sont supérieures à la moyenne.Ensuite, rédige le code en Python correspondant.

Algorithme GestionNotes Variables

somme, moyenne : réel i, N : Entier

Tableau Notes[] : Reel Début

Ecrire("Entrez le nombre de notes :") Lire(N)

Redim(Notes[N])

// Saisie des notes dans le tableau

Pour i de 0 à N-1

Ecrire("Entrez la note numéro ", i + 1, " :") Lire(Notes[i])

FinPour

// Calcul de la somme des notes somme ← 0

Pour i de 0 à N-1

somme ← somme + Notes[i]

FinPour

// Calcul de la moyenne

moyenne ← somme / N

Ecrire("La somme des notes est : ", somme) Ecrire("La moyenne des notes est : ", moyenne)

// Affichage des notes supérieures à la moyenne

Ecrire("Les notes supérieures à la moyenne sont :")

Pour i de 0 à N-1

Si (Notes[i] > moyenne) Alors Ecrire(Notes[i])

FinSi

FinPour

Fin

Python

```
# Lecture du nombre de notes

N = int(input("Entrez le nombre de notes : "))

# Initialisation du tableau pour stocker les notes

Notes = []

# Saisie des notes dans le tableau

for i in range(N):

    note = float(input(f"Entrez la note numéro {i + 1} : "))

    Notes.append(note)  // Notes.append(note) : Ajoute la note saisie dans la liste Notes

# Calcul de la somme des notes

somme = sum(Notes)
```

```
# Calcul de la moyenne

moyenne = somme / N

# Affichage de la somme et de la moyenne

print(f"La somme des notes est : {somme}")

print(f"La moyenne des notes est : {moyenne:.2f}")



# Affichage des notes supérieures à la moyenne

print("Les notes supérieures à la moyenne sont :")

for note in Notes:

    if note > moyenne:

        print(note)
```

Exercice 2

Ecrire un algorithme permettant à l'utilisateur de saisir un nombre quelconque de valeurs et de les stocker dans un tableau. Une fois la saisie est terminée, le programme affichera le nombre de valeurs négatives et le nombre de valeurs positives. Ensuite, rédige le code en Python correspondant.

Exercice 2 :

```
Algorithme nombre_positif_negatif Variables

Nb, Nbpos, Nbneg : Entier

 Tableau T[] : Reel Début

Ecrire("Entrez le nombre de valeurs :") Lire(Nb)

Redim(T[Nb]) Nbpos ← 0

Nbneg ← 0

// Saisie des valeurs dans le tableau et comptage des positifs et négatifs Pour i de 0 à Nb - 1

Ecrire("Entrez le nombre n° ", i + 1) Lire(T[i])

Si (T[i] > 0) Alors Nbpos ← Nbpos + 1

SinonSi (T[i] < 0) Alors Nbneg ← Nbneg + 1

FinSi FinPour

Ecrire("Nombre de valeurs positives : ", Nbpos) Ecrire("Nombre de valeurs négatives : ",
Nbneg)

Fin
```

python

```
# Lecture du nombre de valeurs

Nb = int(input("Entrez le nombre de valeurs : "))

# Initialisation des variables et du tableau

T = []

Nbpos = 0

Nbneg = 0

# Saisie des valeurs dans le tableau et comptage des positifs et négatifs

for i in range(Nb):

    valeur = float(input(f"Entrez le nombre n° {i + 1} : "))

    T.append(valeur)

        if valeur > 0:

        Nbpos += 1  # Incrémenter le compteur des nombres positifs

    elif valeur < 0:

        Nbneg += 1  # Incrémenter le compteur des nombres négatifs

# Affichage des résultats

print(f"Nombre de valeurs positives : {Nbpos}")

print(f"Nombre de valeurs négatives : {Nbneg}")
```

Exercice 3

Ecrire un algorithme constituant un tableau, à partir de deux tableaux de même longueur préalablement saisis. Le nouveau tableau sera la somme des éléments des deux tableaux de départ.Ensuite, rédige le code en Python correspondant.

Exercice 3 :
Algorithme Somme_2_tableaux Variables
i, N : Entier
Tableaux T1[], T2[], T3[] : Reel Début
Pour i de 0 à N - 1 T3[i] ← T1[i] + T2[i]
FinPour Fin

Python .

```
# Lecture de la longueur des tableaux
N = int(input("Entrez la longueur des tableaux : "))

# Initialisation des tableaux
T1 = []
T2 = []
T3 = []

# Saisie des éléments de T1
print("Saisie des éléments du tableau T1 :")
for i in range(N):
    valeur = float(input(f"Entrez l'élément n° {i + 1} de T1 : "))
    T1.append(valeur)

# Saisie des éléments de T2
print("Saisie des éléments du tableau T2 :")
for i in range(N):
    valeur = float(input(f"Entrez l'élément n° {i + 1} de T2 : "))
    T2.append(valeur)

# Calcul des éléments de T3 (somme des éléments de T1 et T2)
for i in range(N):
    T3.append(T1[i] + T2[i])

# Affichage des éléments de T3
print("Les éléments du tableau T3 (somme des deux tableaux) sont :")
for i in range(N):
    print(f"Élément n° {i + 1} : {T3[i]}")
```

Exercice 4

Ecrire un algorithme qui permet la saisie d'un tableau croissant : si T[k]<T[k+1] on enregistre, si T[k]>T[k+1] on redemande la saisie d'un nombre plus grand. Ensuite, rédige le code en Python correspondant.

Exercice 4 :
Algorithme TableauCroissant Variables
i, N : Entier Tableau T[] : Reel
Début
Ecrire ("Entrez la taille du tableau") Lire N
Redim (T[N])
Ecrire (" Saisir le premier nombre") Lire T[0]
Pour i de 1 à N-1 Répéter

Ecrire ("Entrez le nombre numéro ", i + 1) Lire T[i]
Si (T[i] < T[i-1]) Alors
Ecrire (" Vous devez saisir un nombre plus grand") FinSi
Jusqu'à (T[i] >= T[i-1]) FinPour
Fin

Python

```
# Lecture de la taille du tableau
N = int(input("Entrez la taille du tableau : "))

# Initialisation du tableau
T = [0] * N  # Crée un tableau de taille N avec des valeurs initiales à 0

# Saisie du premier élément
T[0] = float(input("Saisir le premier nombre : "))

# Saisie des éléments restants
for i in range(1, N):
    while True:  # Répéter jusqu'à ce que la condition soit respectée
        T[i] = float(input(f"Entrez le nombre numéro {i + 1} : "))
        if T[i] >= T[i - 1]:  # Vérifie si le tableau reste croissant
            break
        else:
            print("Vous devez saisir un nombre plus grand.")

# Affichage du tableau croissant
print("Le tableau croissant saisi est :")
print(T)
```

Exercice 5

Ecrire un algorithme permettant d'inverser l'ordre des éléments dans un tableau de N entiers. Ensuite, rédige le code en Python correspondant.

Exercice 5 :
Algorithme InverseTableau Variables
i, j, N, tmp : Entier Tableau T[] : Entier
Début
Pour i de 0 à (N-1)/2 tmp ← T[i]
T[i] ← T[N-1-i]
T[N-1-i] ← tmp FinPour
Fin

Python

```
# Lecture de la taille du tableau
N = int(input("Entrez la taille du tableau : "))

# Initialisation du tableau et saisie des éléments
T = []
for i in range(N):
    T.append(int(input(f"Entrez l'élément numéro {i + 1} : ")))

# Inversion des éléments du tableau
for i in range(N // 2):  # Parcours jusqu'à la moitié du tableau
    tmp = T[i]  # Stocker l'élément actuel
    T[i] = T[N - 1 - i]  # Échanger avec l'élément opposé
    T[N - 1 - i] = tmp  # Remettre la valeur stockée

# Affichage du tableau inversé
print("Le tableau inversé est :")
print(T)
```

Exercice 6

Ecrire un algorithme qui permet de compter et d'éliminer les zéros à partir d'un tableau de N entiers. Ensuite, rédige le code en Python correspondant.

Exercice 6 :

Algorithme supprimer_zero Variables i, j, N, cpt: Entier

Tableau T[]: Entier Début

cpt ← 0

Pour i de 0 à N-1

Si (T[i] = 0) Alors cpt ← cpt + 1 Pour j de i à N-2

T[j] ← T[j+1]

FinPour

N ← N - 1

Redim (T[N]) i ← i - 1

FinSi FinPour

Ecrire ("Le nombre d'occurrences de 0 supprimées est", cpt) Fin

Python

```
# Lecture de la taille du tableau
N = int(input("Entrez la taille du tableau : "))

# Initialisation du tableau et saisie des éléments
T = []
for i in range(N):
    T.append(int(input(f"Entrez l'élément numéro {i + 1} : ")))

# Suppression des zéros et comptage
cpt = 0  # Compteur des zéros
i = 0    # Indice courant

while i < len(T):  # Parcourir tous les éléments du tableau
    if T[i] == 0:  # Si un zéro est trouvé
        cpt += 1   # Incrémenter le compteur
        T.pop(i)   # Supprimer l'élément courant (décalage automatique)
    else:
        i += 1  # Passer au prochain élément si pas de zéro

# Affichage des résultats
print(f"Le nombre d'occurrences de 0 supprimées est : {cpt}")
print("Le tableau final est :", T)
```

Exercice 7

Ecrire un algorithme qui pour chaque élément d'un tableau T affiche cet élément si son double (au sens mathématique 2x) est aussi présent dans le tableau. Les éléments sont rangés dans un ordre aléatoire à l'intérieur du tableau. Ensuite, rédige le code en Python correspondant.

Exercice 7 :

Algorithme ElementDouble Variables

i, j, N: Entier

Tableau T[]: Entier Début

Pour i de 0 à N-1 Pour j de 0 à N-1

Si (T[j] = 2 * T[i]) Alors Ecrire (T[i])

FinSi FinPour

FinPour Fin

Python

```
# Lecture de la taille du tableau
N = int(input("Entrez la taille du tableau : "))

# Saisie des éléments du tableau
T = [int(input(f"Entrez l'élément numéro {i + 1} : ")) for i in range(N)]

# Parcourir chaque élément du tableau
for i in range(N):
    for j in range(N):
        if T[j] == 2 * T[i]:  # Vérifier si le double existe
            print(f"L'élément {T[i]} a son double {T[j]} dans le tableau.")
            break  # Optionnel : éviter les répétitions inutiles
```

Algorithmique et Programmation (SMP3)
TD2 : Instructions élémentaires
2024-2025

Exercice 4

Ecrire un algorithme qui permet la saisie d'un tableau croissant : si T[k]<T[k+1] on enregistre, si T[k]>T[k+1] on redemande la saisie d'un nombre plus grand. Ensuite, rédige le code en Python correspondant.

Exercice 4 :

```
Algorithme TableauCroissant Variables
i, N : Entier Tableau T[] : Reel
Début
Ecrire ("Entrez la taille du tableau") Lire N
Redim (T[N])
Ecrire (" Saisir le premier nombre") Lire T[0]
Pour i de 1 à N-1 Répéter

Ecrire ("Entrez le nombre numéro ", i + 1) Lire T[i]
Si (T[i] < T[i-1]) Alors
Ecrire (" Vous devez saisir un nombre plus grand") FinSi
Jusqu'à (T[i] >= T[i-1]) FinPour
Fin
```

Python

```
# Lecture de la taille du tableau
N = int(input("Entrez la taille du tableau : "))

# Initialisation du tableau
T = [0] * N  # Crée un tableau de taille N avec des valeurs initiales à 0

# Saisie du premier élément
T[0] = float(input("Saisir le premier nombre : "))

# Saisie des éléments restants
for i in range(1, N):
    while True:  # Répéter jusqu'à ce que la condition soit respectée
        T[i] = float(input(f"Entrez le nombre numéro {i + 1} : "))
        if T[i] >= T[i - 1]:  # Vérifie si le tableau reste croissant
            break
        else:
            print("Vous devez saisir un nombre plus grand.")

# Affichage du tableau croissant
print("Le tableau croissant saisi est :")
print(T)
```

Exercice 5

Ecrire un algorithme permettant d'inverser l'ordre des éléments dans un tableau de N entiers. Ensuite, rédige le code en Python correspondant.

Exercice 5 :

```
Algorithme InverseTableau Variables
i, j, N, tmp : Entier Tableau T[] : Entier
Début
Pour i de 0 à (N-1)/2
tmp ← T[i]
T[i] ← T[N-1-i]
T[N-1-i] ← tmp
FinPour
Fin
```

Python

```
# Lecture de la taille du tableau
N = int(input("Entrez la taille du tableau : "))

# Initialisation du tableau et saisie des éléments
T = []
for i in range(N):
    T.append(int(input(f"Entrez l'élément numéro {i + 1} : ")))

# Inversion des éléments du tableau
for i in range(N // 2):  # Parcours jusqu'à la moitié du tableau
    tmp = T[i]  # Stocker l'élément actuel
    T[i] = T[N - 1 - i]  # Échanger avec l'élément opposé
    T[N - 1 - i] = tmp  # Remettre la valeur stockée

# Affichage du tableau inversé
print("Le tableau inversé est :")
print(T)
```

Exercice 6

Ecrire un algorithme qui permet de compter et d'éliminer les zéros à partir d'un tableau de N entiers. Ensuite, rédige le code en Python correspondant.

Exercice 6 :

```
Algorithme supprimer_zero Variables i, j, N, cpt: Entier
Tableau T[]: Entier Début
cpt ← 0
Pour i de 0 à N-1
Si (T[i] = 0) Alors cpt ← cpt + 1
Pour j de i à N-2
T[j] ← T[j+1]
FinPour
N ← N - 1
Redim (T[N]) i ← i - 1
FinSi FinPour
Ecrire ("Le nombre d'occurrences de 0 supprimées est", cpt) Fin
```

Python

```
# Lecture de la taille du tableau
N = int(input("Entrez la taille du tableau : "))

# Initialisation du tableau et saisie des éléments
T = []
for i in range(N):
    T.append(int(input(f"Entrez l'élément numéro {i + 1} : ")))

# Suppression des zéros et comptage
cpt = 0  # Compteur des zéros
i = 0    # Indice courant

while i < len(T):  # Parcourir tous les éléments du tableau
    if T[i] == 0:  # Si un zéro est trouvé
        cpt += 1   # Incrémenter le compteur
        T.pop(i)   # Supprimer l'élément courant (décalage automatique)
    else:
        i += 1  # Passer au prochain élément si pas de zéro

# Affichage des résultats
print(f"Le nombre d'occurrences de 0 supprimées est : {cpt}")
print("Le tableau final est :", T)
```

Exercice 7

Ecrire un algorithme qui pour chaque élément d'un tableau T affiche cet élément si son double (au sens mathématique 2x) est aussi présent dans le tableau. Les éléments sont rangés dans un ordre aléatoire à l'intérieur du tableau. Ensuite, rédige le code en Python correspondant.

Exercice 7 :

Algorithme ElementDouble Variables

i, j, N: Entier

Tableau T[]: Entier Début

Pour i de 0 à N-1 Pour j de 0 à N-1

Si (T[j] = 2 * T[i]) Alors Ecrire (T[i])

FinSi FinPour

FinPour Fin

Python

```
# Lecture de la taille du tableau
N = int(input("Entrez la taille du tableau : "))

# Saisie des éléments du tableau
T = [int(input(f"Entrez l'élément numéro {i + 1} : "))
 for i in range(N)]

# Parcourir chaque élément du tableau
for i in range(N):
   for j in range(N):
      if T[j] == 2 * T[i]:  # Vérifier si le double existe
         print(f"L'élément {T[i]} a son double {T[j]} dans le tableau.")
         break  # Optionnel : éviter les répétitions inutiles
```

Exercice 8 :

Soit A une matrice de dimensions L et C. Ecrire un algorithme qui permet de:

1. Remplir la matrice A à partir des valeurs saisies au clavier.
2. Calculer la somme des éléments positifs et la somme des éléments négatifs
3. de la matrice A.
4. Déterminer l'élément le plus grand de la matrice A ainsi que sa position.
5. Calculer la transposé de la

matrice A.Ensuite, rédige le code en Python correspondant.

Algorithme OperationMatriceA

Variables i, j, L, C, Spos, Sneg, Max, PosL, PosC : Entier

Tableau A[][], tA[][] : Entier

Début

// Saisie de la matrice A

Ecrire "Donnez le nombre de lignes et de colonnes de la matrice A :" Lire L, C

Redim A[L][C]

Ecrire "Saisie de la matrice A :"

Pour i de 0 à L-1

Pour j de 0 à C-1

Ecrire "Entrez la valeur de la position [", i, ",", j, "] :"

Lire A[i][j]

FinPour

FinPour

// Calcul des sommes des éléments positifs et négatifs

Spos ← 0

Sneg ← 0

Pour i de 0 à L-1

Pour j de 0 à C-1

Si A[i][j] > 0 Alors

Spos ← Spos + A[i][j]

SinonSi A[i][j] < 0 Alors Sneg ← Sneg + A[i][j]

FinSi FinPour

FinPour

Ecrire "La somme des éléments positifs est : ", Spos

Ecrire "La somme des éléments négatifs est : ", Sneg

// Détermination de l'élément le plus grand et sa position Max ← A[0][0]

PosL ← 0

PosC ← 0

Pour i de 0 à L-1

Pour j de 0 à C-1

Si A[i][j] > Max Alors Max ← A[i][j] PosL ← i

PosC ← j FinSi

FinPour FinPour

Ecrire "Le maximum est : ", Max, ". Il se trouve dans la ligne : ", PosL, " et la colonne : ", PosC

// Calcul de la transposée de la matrice A Redim tA[C][L]

Pour i de 0 à C-1 Pour j de 0 à L-1

tA[i][j] ← A[j][i] FinPour

FinPour Fin

Exercice 9
Ecrire un algorithme qui calcule la matrice S somme de deux matrices A et B de dimensions L et C. Ensuite, rédige le code en Python correspondant.

Algorithme SommeMatrice Variable i, j, L, C: entier

Tableau A[][], B[][] , S[][] : entier Debut Pour i =0 à L-1

Pour j = 0 à C-1

S[i][j] ← A[i][j] + B[i][j] FinPour FinPour

Fin

Python

```
# Lecture des dimensions

L = int(input("Entrez le nombre de lignes (L) : "))

C = int(input("Entrez le nombre de colonnes (C) : "))

# Initialisation des matrices

A = []

B = []

S = [[0 for _ in range(C)] for _ in range(L)]

# Saisie de la matrice A

print("Remplissez la matrice A :")

for i in range(L):

    row = []

    for j in range(C):

        row.append(int(input(f"Entrez A[{i}][{j}] : ")))
```

```
    A.append(row)

# Saisie de la matrice B

print("Remplissez la matrice B :")

for i in range(L):

    row = []

    for j in range(C):

        row.append(int(input(f"Entrez B[{i}][{j}] : ")))

    B.append(row)

# Calcul de la matrice somme S

for i in range(L):

    for j in range(C):

        S[i][j] = A[i][j] + B[i][j]

# Affichage de la matrice S

print("La matrice somme S est :")
```

Exercice 10
Ecrire un algorithme qui construit et affiche une matrice carrée unitaire U de dimension N. Ensuite, rédige le code en Python correspondant.

```
Algorithme VecteurMatrice

Variable  i, j, N: entier

Tableau U[][]: entier

Debut

// Saisie de la dimension de la matrice

Ecrire ("Saisir la dimension de la matrice :") Lire (N)

Redim( U[N][N])

// Initialisation de la matrice

Pour i de 0 à N-1

Pour j de 0 à N-1

 Si i = j Alors U[i][j] ← 1

Sinon

U[i][j] ← 0 FinSi

FinPour FinPour

// Affichage de la matrice Pour i de 0 à N-1

Pour j de 0 à N-1

Ecrire (U[i][j], " " )

FinPour Ecrire ("\n")

FinPour Fin

Python

def creer_matrice_unitaire(N):

    # Initialisation de la matrice carrée de taille N

    U = [[0 for _ in range(N)] for _ in range(N)]

        # Remplir la matrice unitaire

    for i in range(N):

        for j in range(N):

            if i == j:

                U[i][j] = 1

            else:

                U[i][j] = 0


    # Affichage de la matrice
```

```
    for i in range(N):
        for j in range(N):
            print(U[i][j], end=" ")
        print()

# Saisie de la dimension de la matrice
N = int(input("Saisir la dimension de la matrice : "))

# Appel de la fonction pour créer et afficher la matrice unitaire
creer_matrice_unitaire(N)
```

Exercice 11
Ecrire un algorithme qui transfère un tableau M à deux dimensions L et C dans un tableau V à une dimension L*C. Ensuite, rédige le code en Python correspondant.

```
Algorithme VecteurMatrice Variables i, j, k, L, C: entier Tableau M[][], V[]: entier
Debut
Ecrire ("Donner le nombre de lignes et le nombre de colonnes de la matrice M :") Lire (L, C)
Redim (M[L][C])
Ecrire ("Saisie de la matrice M :")
Pour i de 0 à L-1
Pour j de 0 à C-1
Ecrire ("Donner la valeur de la position (", i, ", ", j, ") :") Lire (M[i][j])
FinPour FinPour
Redim (V[L*C])
k ← 0
Pour i de 0 à L-1 Pour j de 0 à C-1
V[k] ← M[i][j] k ← k + 1
FinPour FinPour
Fin
```

```
def transferer_matrice_vers_vecteur(L, C):
    # Initialisation de la matrice M avec les dimensions L x C
    M = []
    print("Saisie de la matrice M :")
    for i in range(L):
        ligne = []
        for j in range(C):
            # Saisie des éléments de la matrice M
            valeur = int(input(f"Donner la valeur de la position ({i}, {j}) : "))
            ligne.append(valeur)
        M.append(ligne)
    # Initialisation du vecteur V à une dimension L * C
    V = []
    k = 0
    # Transfert des éléments de M dans V
    for i in range(L):
        for j in range(C):
            V.append(M[i][j])
            k += 1
    # Affichage du vecteur V
    print("Le tableau V à une dimension est :", V)
# Saisie des dimensions de la matrice
L = int(input("Donner le nombre de lignes de la matrice M : "))
```

```
C = int(input("Donner le nombre de colonnes de la matrice M : "))

# Appel de la fonction pour transférer la matrice dans le vecteur

transferer_matrice_vers_vecteur(L, C)
```

Exercice 12

Ecrire un algorithme qui permet de faire la permutation de deux matrices dont les valeurs sont saisies à partir du clavier. Afficher les matrices avant et après permutation (taille des matrices aux choix). Ensuite, rédige le code en Python correspondant.

```
Algorithme Echange_matrice Variables
i, j, k, l, n, m, Z : Entier Tableau T1[][], T2[][] : Entier
Début
Ecrire ("Donner le nombre de lignes et le nombre de colonnes des matrices :") Lire (n, m)
Redim (T1[n][m])
Redim (T2[n][m])
Ecrire ("Saisie de la matrice T1 :") Pour i de 0 à n-1
Pour j de 0 à m-1
Ecrire ("Donner la valeur de la position (", i, ", ", j, ") :") Lire (T1[i][j])
FinPour FinPour
Ecrire ("Saisie de la matrice T2 :") Pour i de 0 à n-1
Pour j de 0 à m-1
Ecrire ("Donner la valeur de la position (", i, ", ", j, ") :") Lire (T2[i][j])
FinPour FinPour
Ecrire ("La matrice T1 avant permutation :") Pour i de 0 à n-1
Pour j de 0 à m-1 Ecrire (T1[i][j])
FinPour Ecrire ("\n")
FinPour
Ecrire ("La matrice T2 avant permutation :") Pour i de 0 à n-1
Pour j de 0 à m-1 Ecrire (T2[i][j])
FinPour Ecrire ("\n")
FinPour
Pour i de 0 à n-1 Pour j de 0 à m-1
Z ← T1[i][j]
 T1[i][j] ← T2[i][j]
T2[i][j] ← Z
FinPour

FinPour

Ecrire ("La matrice T1 après permutation :") Pour i de 0 à n-1
Pour j de 0 à m-1 Ecrire (T1[i][j])
FinPour Ecrire ("\n")
FinPour
Ecrire ("La matrice T2 après permutation :") Pour i de 0 à n-1
Pour j de 0 à m-1 Ecrire (T2[i][j])
FinPour Ecrire ("\n")
FinPour Fin
```

Python

```
def echanger_matrices():
    # Saisie des dimensions des matrices
    n = int(input("Donner le nombre de lignes des matrices : "))
    m = int(input("Donner le nombre de colonnes des matrices : "))

    # Initialisation des matrices T1 et T2
    T1 = []
    T2 = []

    # Saisie des éléments de la matrice T1
    print("Saisie de la matrice T1 :")
    for i in range(n):
        ligne = []
        for j in range(m):
            valeur = int(input(f"Donner la valeur de la position ({i}, {j}) : "))
            ligne.append(valeur)
        T1.append(ligne)

    # Saisie des éléments de la matrice T2
    print("Saisie de la matrice T2 :")
    for i in range(n):
        ligne = []
        for j in range(m):
            valeur = int(input(f"Donner la valeur de la position ({i}, {j}) : "))
            ligne.append(valeur)
        T2.append(ligne)

    # Affichage des matrices avant la permutation
    print("\nLa matrice T1 avant permutation :")
    for i in range(n):
        print(" ".join(map(str, T1[i])))

    print("\nLa matrice T2 avant permutation :")
    for i in range(n):
        print(" ".join(map(str, T2[i])))

    # Permutation des matrices
    for i in range(n):
        for j in range(m):
            T1[i][j], T2[i][j] = T2[i][j], T1[i][j]

    # Affichage des matrices après la permutation
    print("\nLa matrice T1 après permutation :")
    for i in range(n):
        print(" ".join(map(str, T1[i])))

    print("\nLa matrice T2 après permutation :")
    for i in range(n):
        print(" ".join(map(str, T2[i])))

# Appel de la fonction
echanger_matrices()
```

Exercice 13

Écrivez deux fonctions prenant en entrée un tableau de 100 entiers saisis par l'utilisateur. Une fonction doit déterminer la position du plus petit élément (noté posmin) et la position du deuxièmedoit déterminer le plus grand élément (noté posmax) de ce tableau.

Fonction trouver_position_min(Tableau tableau[100] : entiers): entier Début
posmin ← 0
min_val ← tableau[0]
Pour i ← 1 à 99
Si tableau[i] < min_val Alors min_val ← tableau[i] posmin ← i
FinSi FinPour
Retourner posmin Fin
Fonction trouver_position_max (Tableau tableau[100] : entiers): entier Début
posmax ← 0
max_val ← tableau[0]
Pour i ← 1 à 99
Si tableau[i] > max_val Alors max_val ← tableau[i] posmax ← i
FinSi FinPour
Retourner posmax Fin

Python

```
# Fonction pour trouver la position du plus petit élément
def trouver_position_min(tableau):
    posmin = 0
    min_val = tableau[0]
    for i in range(1, 100):
        if tableau[i] < min_val:
            min_val = tableau[i]
            posmin = i
    return posmin

# Fonction pour trouver la position du plus grand élément
def trouver_position_max(tableau):
    posmax = 0
    max_val = tableau[0]
    for i in range(1, 100):
        if tableau[i] > max_val:
            max_val = tableau[i]
            posmax = i
    return posmax

# Demander à l'utilisateur de saisir 100 entiers
tableau = []
print("Veuillez entrer 100 entiers :")
for i in range(100):
    val = int(input(f"Entier {i+1}: "))
    tableau.append(val)

# Appeler les fonctions et afficher les résultats
posmin = trouver_position_min(tableau)
posmax = trouver_position_max(tableau)

print(f"Le plus petit élément se trouve à la position {posmin} (valeur: {tableau[posmin]})")
print(f"Le plus grand élément se trouve à la position {posmax} (valeur: {tableau[posmax]})")
```

Exercice 14

Soit T un tableau de N notes saisis par l'utilisateur. Écrire le programme en utilisant lesfonctions qui permet de :
1. Calculer la somme des éléments de ce tableau, ainsi que leur moyenne.
2. Afficher les notes qui sont supérieures à la moyenne.

Algorithme GestionNotes
Variables som, moy : réel i, N : Entier
Tableau Notes[] : Reel Début
Ecrire ("Entrez le nombre de notes") Lire (N)
Redim (Notes[N])
Pour i = 0 A N-1
Ecrire ("Entrez la note numéro ", i + 1) Lire Notes[i]
FinPour
//
Fonction calculerSomme(tableau tableau[N] : réels): réel Début
somme ← 0
Pour i = 0 A N-1
somme ← somme + tableau[i] FinPour
Retourner somme Fin
Fonction calculerMoyenne(tableau tableau[N] : réels): réel Début
somme ← calculerSomme(tableau)
moyenne ← somme / N
Retourner moyenne Fin
Procedure afficherNotesSuperieuresMoyenne (tableau tableau[N] : réels): réel Début
moyenne ← calculerMoyenne(tableau)
Ecrire (" Les notes superieures à la moyenne sont :")
Pour i = 0 A N-1
Si tableau[i]> moyenne Alors Ecrire (tableau[i] , " ")
FinSi FinPour
Fin
//
som ← calculerSomme(Notes)
moy ← calculerMoyenne (Notes)
afficherNotesSuperieuresMoyenne (Notes)
Fin

Python

```
# Fonction pour calculer la somme des éléments d'un tableau
def calculerSomme(tableau):
    somme = 0
    for note in tableau:
        somme += note
    return somme

# Fonction pour calculer la moyenne des éléments d'un tableau
def calculerMoyenne(tableau):
    somme = calculerSomme(tableau)
    moyenne = somme / len(tableau)
    return moyenne

# Procédure pour afficher les notes supérieures à la moyenne
def afficherNotesSuperieuresMoyenne(tableau):
    moyenne = calculerMoyenne(tableau)
    print(f"Les notes supérieures à la moyenne ({moyenne:.2f}) sont :")
    for note in tableau:
        if note > moyenne:
```

```
            print(note, end=" ")
        print()  # Pour une nouvelle ligne après l'affichage des notes

    # Programme principal
    print("Entrez le nombre de notes :")
    N = int(input())
    Notes = []

    # Saisie des notes
    for i in range(N):
        note = float(input(f"Entrez la note numéro {i+1} : "))
        Notes.append(note)

    # Calcul de la somme et de la moyenne
    somme = calculerSomme(Notes)
    moyenne = calculerMoyenne(Notes)

    # Affichage des résultats
    print(f"La somme des notes est : {somme:.2f}")
    print(f"La moyenne des notes est : {moyenne:.2f}")
    afficherNotesSuperieuresMoyenne(Notes)
```

Exercice 15

Ecrire un algorithme permettant d'inverser l'ordre des éléments dans un tableau de N entiers. En utilisant une procédure. Ensuite rédige le code python correspondant

```
Algorithme InverserTableau
Variables
  i, N : Entier
  Tableau T[N] : Entier

Procédure Inverser Tableau (tableau tableau[N] : Entiers)
  Début
    N ← longueur(tableau)
    Pour i de 0 à (N-1) / 2 Faire
      temp ← tableau[i]
      tableau[i] ← tableau[N-1-i]
      tableau[N-1-i] ← temp
    FinPour
  FinProcédure

Début
  Ecrire("Entrez la taille du tableau :")
  Lire(N)
  Redim(T[N])
  Pour i de 0 à N-1 Faire
    Ecrire("Entrez l'élément ", i + 1, ":")
    Lire(T[i])
  FinPour
  InverserTableau(T)
  Ecrire("Le tableau inversé est :")
  Pour i de 0 à N-1 Faire
    Ecrire(T[i], " ")
  FinPour
Fin
```

```
# Définir la procédure d'inversion
def inverser_tableau(tableau):
    n = len(tableau)
    for i in range(n // 2):
        # Échanger l'élément courant avec son opposé
        temp = tableau[i]
        tableau[i] = tableau[n - 1 - i]
        tableau[n - 1 - i] = temp

# Programme principal
print("Entrez la taille du tableau :")
N = int(input())
T = []

# Saisie des éléments du tableau
for i in range(N):
    element = int(input(f"Entrez l'élément {i + 1} : "))
    T.append(element)

# Affichage avant inversion
print("Tableau avant inversion :", T)

# Inversion du tableau
inverser_tableau(T)

# Affichage après inversion
print("Tableau après inversion :", T)
```

Exercice 16

Écrire une procédure qui prend en entrée deux tableaux d'entiers de même taille et qui remplacele premier tableau par la somme des éléments correspondants des deux tableaux.

```
Algorithme SommeTableaux
Variables
  i, N : Entier
  Tableau A[N], B[N] : Entier

Procédure RemplacerParSomme(tableau1 tableau[N], tableau2 tableau[N])
  Début
    Pour i de 0 à N-1 Faire
      tableau1[i] ← tableau1[i] + tableau2[i]
    FinPour
  FinProcédure

Début
  Ecrire("Entrez la taille des tableaux :")
  Lire(N)
  Redim(A[N])
  Redim(B[N])

  Ecrire("Saisie des éléments du tableau A :")
  Pour i de 0 à N-1 Faire
    Ecrire("Entrez l'élément ", i + 1, ":")
    Lire(A[i])
  FinPour
```

```
    Ecrire("Saisie des éléments du tableau B :")
    Pour i de 0 à N-1 Faire
      Ecrire("Entrez l'élément ", i + 1, ":")
      Lire(B[i])
    FinPour

    Appeler RemplacerParSomme(A, B)

    Ecrire("Le tableau A après remplacement :")
    Pour i de 0 à N-1 Faire
      Ecrire(A[i], " ")
    FinPour
  Fin
```

Python

```
# Définir la procédure

def remplacer_par_somme(tableau1, tableau2):

    for i in range(len(tableau1)):

        tableau1[i] += tableau2[i]  # Ajouter les éléments correspondants

# Programme principal

print("Entrez la taille des tableaux :")

N = int(input())

A = []

B = []

# Saisie des éléments du tableau A

print("Saisie des éléments du tableau A :")

for i in range(N):

    element = int(input(f"Entrez l'élément {i + 1} pour A : "))

    A.append(element)

# Saisie des éléments du tableau B

print("Saisie des éléments du tableau B :")

for i in range(N):

    element = int(input(f"Entrez l'élément {i + 1} pour B : "))

    B.append(element)

# Affichage avant modification

print("Tableau A avant modification :", A)

print("Tableau B :", B)

# Appeler la procédure

remplacer_par_somme(A, B)

# Affichage après modification

print("Tableau A après modification :", A)
```

Utilisation

- **Fonction :**
  - Utilisée lorsqu'un calcul ou un traitement doit produire un résultat réutilisable ailleurs.
  - Exemple : Calculer la moyenne, vérifier une condition, transformer des données.
- **Procédure :**
  - Utilisée pour exécuter une séquence d'instructions qui modifient des données ou produisent un effet (affichage, écriture dans un fichier, etc.).
  - Exemple : Afficher un message, remplir une structure de données.

4. Retour de valeurs

- **Fonction :**
  - Retourne une valeur via une instruction return.
- **Procédure :**
  - Ne retourne pas de valeur directement, mais elle peut affecter des variables globales ou des paramètres modifiables.

5. Concept de pureté

- **Fonction :**
  - Peut être une fonction "pure", c'est-à-dire qu'elle n'a pas d'effets secondaires (elle ne modifie pas l'état externe, comme des variables globales).
- **Procédure :**

- A souvent des effets secondaires (modification de variables ou structures externes).

Résumé des différences dans un tableau

Aspect	Fonction	Procédure
Retourne une valeur	Oui (return)	Non
Effets secondaires	Optionnels (fonction pure ou non)	Oui (souvent)
Utilisation	Calculs, transformations	Actions, affichages, modifications
Appelée comme	Partie d'une expression	Instruction autonome